CozyStudy: Project Report

A gamified, anti-cheat study sanctuary built in Python with a functional-programming core. Software design, implementation and verification report.

Report date: 16 October 2026. Source: the 17 project files supplied (main.py, fp_models.py, fp_utils.py, fp_reducers.py, fp_analytics.py, storage.py, room_canvas.py, timer_view.py, charts_canvas.py, task_view.py, calendar_view.py, wardrobe_dialog.py, theme.py, test_app.py, run.sh, README.md, cozystudy_data.json).

Document Control

Item	Detail
Project name	CozyStudy: Aesthetic Gamified Study Sanctuary
Platform	Desktop (macOS, Linux, Windows)
Language and toolkit	Python 3, Tkinter with Canvas drawing
Third-party dependencies	None (standard library only)
Programming paradigm	Functional core, imperative GUI shell
Persistence	Single JSON file (cozystudy_data.json)
Automated tests	6 unittest cases in test_app.py
Report type	Reverse-engineered design, implementation and test report

This report was written by reading the delivered source code, the README and the saved data file. Statements about behaviour are drawn from the code itself. Where the code and the README disagree, or where the code contains a defect, the report says so explicitly in Chapter 9 and Chapter 10 rather than repeating the README's claim.

Report Roadmap

1. Executive Summary
2. Introduction

3. Project Lifecycle and Phases
4. Phase 1: Requirements Analysis
5. Phase 2: System Architecture and Design
6. Phase 3: Back-End Design (Functional Core and Persistence)
7. Phase 4: Front-End Design (Tkinter Views)
8. Phase 5: Implementation Details by Module
9. Phase 6: Testing and Verification
10. Defects, Risks and Code-Review Findings
11. Phase 7: Deployment, Operation and Maintenance
12. Future Work and Conclusion

Appendices: data dictionary, test case catalogue, requirement traceability matrix, glossary.

1. Executive Summary

CozyStudy is a native desktop application that helps students study with focus and rewards them for doing so honestly. It combines five ideas in one window: a Pomodoro-style focus timer, an anti-cheat engine that measures how much of a session the student actually spent looking at the app, a task and deadline manager, analytics charts and a calendar of past work, and a decorative "cozy room" that the student fills with charms bought using a virtual currency called Star Charms.

The application is implemented entirely with the Python standard library. The user interface uses Tkinter widgets and Canvas vector drawing, so there is nothing to install beyond Python itself. A launcher script (run.sh) and a single command (python3 main.py) start it.

The most distinctive engineering decision is the separation of a **pure functional core** from an **imperative graphical shell**. All domain data lives in frozen (immutable) dataclasses. Every change to the application's data is expressed as a pure reducer function that takes the old state and returns a new state. The graphical views never change data directly. They raise events to the main controller, which calls a reducer, saves the result to disk, and pushes the new state to every view. This pattern is similar to the Redux model used in web front ends and makes the business rules easy to test without opening a window.

Key facts at a glance

Measure	Value
Python modules	14 (.py files)
Immutable domain models	7 frozen dataclasses (Subject, Task, Charm, FocusSession, Settings, UserStats, AppState)
Pure reducer functions	9 public plus 1 private helper
Functional utility combinators	13 functions in fp_utils
Analytics pipelines	6 functions in fp_analytics
GUI screens	6 tabs plus 2 dialogs (wardrobe, add task)
Themes	4 (Sakura, Matcha, Lavender, Midnight)
Unlockable room charms	12 across 4 categories
Automated test cases	6

Principal findings

- The architecture is clean and consistent. Data flows one way (view to controller to reducer to storage to views) and the functional core has no dependency on Tkinter.
- The anti-cheat design is well suited to its purpose. It offers an "honest" mode that records distraction without punishing it, and a "strict" mode that pauses the timer when the window loses focus.
- The automated test suite covers the functional core only. It does not exercise the GUI, the unlock and session reducers, or the strict-mode behaviour, although the test file's own docstring says it does.
- Code review found several genuine defects. The most important is that the study-streak date is silently reset on every load (Section 10.1), which the saved data file confirms. Others include an out-of-range index when focusing a task after a completed task (10.2) and a Settings control for the anti-cheat mode that does not reach the timer (10.3).

2. Introduction

2.1 Background

Students increasingly study on laptops, which is also where distraction lives. A student who opens a timer app can leave it running while browsing a social feed, and most timers cannot tell the difference. Gamified study tools try to solve this with streaks and rewards, but they usually reward elapsed time, which encourages cheating the timer rather than focusing.

CozyStudy takes a different approach. It rewards focus quality, not just duration. It measures the fraction of a session during which the app window was in the foreground, calls that the **Focus Integrity Rate**, and scales the reward by it. The visual theme is deliberately calm and cosy, with pastel palettes, a sleeping cat, a steaming cup of matcha and fairy lights, to make the study session feel like a pleasant place to be rather than a chore.

2.2 Problem statement

The project addresses the following problem: *How can a desktop study tool motivate sustained, honest focus, give the student clear feedback on how well they are really focusing, and organise their coursework, while remaining simple to install and easy to reason about and test?*

The problem breaks into four sub-problems.

1. **Measurement.** Detect, with only the standard library, when the student has switched away from the app, and account for that time separately from focused time.
2. **Motivation.** Convert focus quality into a virtual reward and give the student a meaningful place to spend it.
3. **Organisation.** Let the student record tasks with subjects, priorities and deadlines, and connect those tasks to study sessions.
4. **Insight.** Present history as charts, a calendar and key figures so the student can see patterns.

2.3 Aim and objectives

The aim is to deliver a complete, installable-in-seconds desktop application that answers the problem statement. The objectives, each of which is traced to implementation later in the report, are:

ID	Objective
O1	Provide a 25-minute focus timer with 5-minute and 15-minute break modes.
O2	Track active focus time, off-window time and window-switch count for each session, and compute an integrity percentage.
O3	Offer two anti-cheat modes: honest (record only) and strict (pause on leaving).
O4	Award Star Charms scaled by integrity and session length.
O5	Provide an illustrated, animated study room whose contents depend on which charms are unlocked and equipped.
O6	Provide a wardrobe shop where charms are bought and equipped.
O7	Provide a prioritised task manager with deadline indicators and a one-click link to the timer.
O8	Provide analytics: a donut chart, a seven-day bar chart, a daily timeline and summary cards.
O9	Provide a monthly calendar with a per-day inspector.
O10	Persist all data locally and allow backup and reset.
O11	Offer four visual themes.
O12	Build the business logic as immutable data and pure functions, with automated tests.
O13	Require no third-party packages.

2.4 Scope

In scope: a single-user, single-machine desktop application; local JSON storage; the six screens and two dialogs described in this report; the functional core and its unit tests.

Out of scope: user accounts, cloud sync, mobile or web versions, real audio playback of lo-fi music (the radio charm is decorative), true operating-system level application blocking, and multi-device streak sharing.

2.5 Target users

The application is aimed at secondary and university students. Three personas capture the range.

Persona	Description	Main goals	Features that matter most
Maya, 19, computer science undergraduate	Studies in long evening blocks on a laptop with many browser tabs open.	See honestly how much of a session was real focus.	Honest mode, integrity percentage, analytics
Daniel, 22, final-year biology student	Has several deadlines at once and tends to lose track of dates.	Never miss a deadline; know what to study next.	Task manager, deadline badges, Focus Now
Priya, 16, high-school student	Easily distracted; motivated by rewards and decoration.	Earn rewards and personalise a space.	Star Charms, wardrobe, cosy room, streak badge

2.6 Source material reviewed

The table lists every file reviewed and its role.

File	Layer	Role
main.py	Controller and shell	Application window, navigation, event wiring, settings screen, reducer callbacks
fp_models.py	Core	Frozen dataclasses for all domain data
fp_utils.py	Core	Pure combinators and focus and reward formulas
fp_reducers.py	Core	Pure state transitions

File	Layer	Role
fp_analytics.py	Core	Pure aggregation pipelines for charts and statistics
storage.py	Persistence	Seed data, JSON serialisation, load and save
theme.py	Presentation	Colour palettes and font tokens
room_canvas.py	View	Cosy room diorama and animation
timer_view.py	View	Focus timer and anti-cheat tracker
charts_canvas.py	View	Analytics charts
task_view.py	View	Task manager and add-task dialog
calendar_view.py	View	Monthly calendar and day inspector
wardrobe_dialog.py	View	Charm shop and equip dialog
test_app.py	Test	Automated unit tests
run.sh	Operations	Launcher script
README.md	Documentation	User-facing description
cozystudy_data.json	Data	Saved application state (evidence of real use)

2.7 How to read this report

Readers interested in the **business and learning outcomes** should read Chapters 1 to 4. Readers interested in **how the software is built** should read Chapters 5 to 8. Readers interested in **quality** should read Chapters 9 and 10. Chapter 11 covers running and maintaining the software. Diagrams use a simple notation: boxes are modules or components, arrows show the direction of calls or data.

3. Project Lifecycle and Phases

The project is described here as a sequence of seven phases. The code shows the outcome of each phase, and the phases overlap in practice, but separating them makes the engineering reasoning clear. The mapping from phase to deliverable is derived from the files supplied.

Phase	Name	Main activities	Principal deliverables
1	Requirements analysis	Identify users, goals, features, constraints	Feature list in README, objectives O1 to O13
2	System architecture and design	Choose layering, paradigm, data flow, module boundaries	Module structure, one-way data-flow pattern
3	Back-end design	Model data, define reducers, analytics, persistence	fp_models, fp_utils, fp_reducers, fp_analytics, storage
4	Front-end design	Design screens, theme, canvas drawings, interactions	theme, room_canvas, timer_view, charts_canvas, task_view, calendar_view, wardrobe_dialog
5	Implementation and integration	Wire views to controller, build settings screen, seed data	main.py, storage defaults, run.sh
6	Testing and verification	Unit tests for the functional core, data checks	test_app.py, saved-data inspection
7	Deployment and maintenance	Launcher, backup, reset, documentation	run.sh, README, Settings backup and reset

3.1 Phase dependencies

Phases 3 and 4 depend on the architecture chosen in Phase 2 and can proceed in parallel, because the view layer only needs the *shape* of the state (the model classes) and the callback signatures, not the reducer internals. Phase 5 needs both. Phase 6 can begin as soon as Phase 3 exists, because the functional core has no GUI dependency. This is one of the practical benefits of the chosen architecture: the back end could be tested and finished while the front end was still being drawn.

3.2 Phase timeline (indicative)

The delivered files contain no schedule, so the following is an indicative effort split inferred from the size and complexity of each layer, not a recorded timeline.

Phase	Relative effort	Reasoning
1 Requirements	8%	A compact feature set, stated clearly in the README
2 Architecture	10%	Pattern is small but consequential
3 Back end	22%	Models, reducers, analytics, storage, formulas
4 Front end	35%	Six screens, canvas drawings, four themes, animation
5 Integration	10%	Controller wiring and settings screen
6 Testing	8%	Six unit tests; GUI untested
7 Deployment	7%	Launcher, README, backup and reset

4. Phase 1: Requirements Analysis

4.1 Elicitation approach

Requirements for a personal study tool come mainly from the designer's own experience as a student and from observation of existing products such as Pomodoro timers, Forest-style focus apps and to-do managers. The README states the intent in user terms, including the observation that a timer should keep running when you open other tabs, "like you open other tabs but it still goes on". That sentence is the origin of the **honest tracking mode**: the timer must not stop merely because the student looked elsewhere, but it must measure how long they looked elsewhere.

4.2 User stories

ID	As a...	I want to...	So that...
US-01	student	start a 25-minute focus timer	I study in structured blocks
US-02	student	take a short or long break from the same screen	I rest without leaving the app

ID	As a…	I want to…	So that…
US-03	student	see how much of a session I was truly focused	I understand my habits honestly
US-04	student	choose whether leaving the window pauses my timer	I can pick gentle or strict discipline
US-05	student	earn rewards for focusing well	I stay motivated
US-06	student	spend rewards on room decorations	I have something to look forward to
US-07	student	record tasks with a subject, priority and due date	I know what to study
US-08	student	see which tasks are overdue or due today	I never miss a deadline
US-09	student	start a session already linked to a task	progress is counted for me
US-10	student	mark a task done and get a bonus if it is on time	I am rewarded for finishing early
US-11	student	see charts of my focus over the last week	I can spot trends
US-12	student	browse a calendar and open any past day	I can review what I did
US-13	student	change the colour theme	the app suits my taste or lighting
US-14	student	back up and reset my data	I do not lose work and can start over
US-15	student	click on my cat and my lamp	the room feels alive

4.3 Functional requirements

ID	Requirement	Objective	Implemented in
FR-01	The system shall provide a focus mode with a default of 25 minutes.	O1	Settings.pomodoro_minutes, timer_view
FR-02	The system shall provide a short break (5 minutes) and a long break (15 minutes).	O1	Settings, timer_view._switch_timer_mode
FR-03	The system shall display remaining time as MM:SS inside a circular progress ring.	O1	fp_utils.format_seconds_to_min_sec, timer_view._draw_timer_ring
FR-04	The system shall allow starting, pausing, resuming and resetting a session.	O1	timer_view._toggle_timer, _reset_timer
FR-05	The system shall count seconds spent with the window focused as active focus.	O2	timer_view._timer_tick_loop
FR-06	The system shall count seconds spent with the window unfocused as distracted time.	O2	timer_view._timer_tick_loop
FR-07	The system shall count window-focus losses as tab switches.	O2	timer_view.on_window_focus_out
FR-08	The system shall compute the integrity rate as active seconds divided by elapsed seconds, as a percentage.	O2	fp_utils.calculate_focus_integrity

ID	Requirement	Objective	Implemented in
FR-09	In strict mode the system shall pause the session when the window loses focus.	O3	timer_view.on_window_focus_out
FR-10	The system shall play an alert when focus is lost and a chime when a session ends, if sound is enabled.	O3	timer_view._play_sound_alert, _play_completion_sound
FR-11	The system shall award Star Charms on completing a focus session, scaled by integrity.	O4	fp_utils.calculate_charm_reward
FR-12	The system shall record each completed session as an immutable FocusSession.	O2, O12	timer_view._handle_timer_completed, fp_reducers.add_session
FR-13	The system shall increment a linked task's completed cycles when a session finishes.	O7	fp_reducers.add_session
FR-14	The system shall maintain a daily study streak and a best streak.	O4	fp_reducers._compute_streak
FR-15	The system shall draw a room with wall, floor, rug, window, bookshelf, desk, chair and laptop.	O5	room_canvas.redraw
FR-16	The system shall draw each equipped charm in the room and omit unequipped ones.	O5	room_canvas._draw_equipped_charms

ID	Requirement	Objective	Implemented in
FR-17	The system shall animate steam, the cat, fairy lights, clouds, stars and floating objects.	O5	room_canvas._start_animation_loop
FR-18	The system shall let the user click the cat for a message and the lamp to toggle its light.	O5	room_canvas._on_click
FR-19	The system shall list charms by category with name, icon, description and cost.	O6	wardrobe_dialog
FR-20	The system shall unlock a charm only when the balance covers its cost, and deduct the cost.	O6	fp_reducers.unlock_charm
FR-21	The system shall toggle equipped state for unlocked charms.	O6	fp_reducers.toggle_equip_charm
FR-22	The system shall allow adding a task with title, subject, priority, due date, estimated cycles and notes.	O7	task_view._open_add_task_dialog, fp_reducers.add_task
FR-23	The system shall filter tasks by priority and by active or completed status.	O7	task_view._set_filter
FR-24	The system shall label deadlines as overdue, due today or due on a date.	O7	task_view._render_task_card
FR-25	The system shall provide a Focus Now action that binds a task to the timer.	O7	task_view, main._on_focus_task

ID	Requirement	Objective	Implemented in
FR-26	The system shall complete a task and award a bonus, larger if on time and if high priority.	O7	fp_reducers.complete_task
FR-27	The system shall delete a task after confirmation.	O7	task_view._confirm_delete, fp_reducers.delete_task
FR-28	The system shall show summary cards, a donut chart, a seven-day bar chart and a daily timeline.	O8	charts_canvas, fp_analytics
FR-29	The system shall show a month grid with navigation and activity dots, and a day inspector.	O9	calendar_view
FR-30	The system shall persist state to JSON after every change and reload it on start.	O10	storage.save_state, load_state
FR-31	The system shall export a backup and reset to demo data.	O10	main._export_backup, _reset_demo
FR-32	The system shall provide four themes selectable at runtime.	O11	theme.THEMES, main._on_theme_change
FR-33	The system shall switch tabs with keys 1 to 5.	O1 to O9	main._bind_global_events

4.4 Non-functional requirements

ID	Category	Requirement	How it is met
NFR-01	Portability	Run on macOS, Linux and Windows with Python 3 and Tkinter.	Standard library only; run.sh falls back to a macOS framework path

ID	Category	Requirement	How it is met
NFR-02	Installability	No package installation step.	Zero third-party imports
NFR-03	Maintainability	Business rules separable from the GUI.	Functional core with no tkinter import
NFR-04	Testability	Business rules testable without a display.	Pure functions and frozen data
NFR-05	Reliability	Corrupt or missing data must not crash start-up.	try/except in load_state and dict_to_state with default fallback
NFR-06	Data integrity	A crash mid-write should not lose all data.	Not met; the file is overwritten in place (Section 10.6)
NFR-07	Usability	Consistent visual language and clear status feedback.	Shared theme tokens, coloured pills and badges
NFR-08	Performance	UI stays responsive on a typical laptop.	Small canvases; room redraws at 10 frames per second
NFR-09	Privacy	Data stays on the user's machine.	Local JSON, no network calls
NFR-10	Accessibility	Readable contrast and text sizes.	Partially met; fixed small fonts (9 pt) and colour-only cues in places
NFR-11	Aesthetics	A calm, pastel look that encourages use.	Four hand-tuned palettes

4.5 Constraints and assumptions

Constraints. The application must use only the Python standard library. Tkinter is the only available GUI toolkit under that constraint. Tkinter has no native charting, so the charts are drawn manually with Canvas primitives. Tkinter offers no portable API to detect whether a particular other application or browser tab is open, so anti-cheat measures window focus rather than content.

Assumptions.

- The student runs the app on the same machine on which they study.

- Losing window focus is a reasonable proxy for distraction. (A student could look at a book while the app stays focused; the system cannot know.)
- Sound uses macOS's afplay when available and the Tk bell otherwise.
- One user per data file.
- The system clock is correct; all dates derive from the local clock.

4.6 Risks identified at requirements stage

Risk	Likelihood	Impact	Mitigation chosen
Focus-loss detection is a weak proxy for distraction	High	Medium	Present as "integrity" not "proof"; offer honest mode that does not punish
Reward inflation makes the shop trivial	Medium	Medium	Costs 30 to 160; rewards scale with integrity
Data loss on crash	Low	High	JSON saved after every change (but see NFR-06)
Theme or font differences across operating systems	Medium	Low	Helvetica fallback via Tk
Emoji rendering varies by platform	Medium	Low	Emoji used as icons only, never as the sole carrier of meaning

4.7 Requirements-to-module overview

Module	Requirements primarily satisfied
fp_models	FR-12, FR-14, FR-20, FR-22 (data shape)
fp_utils	FR-03, FR-08, FR-11
fp_reducers	FR-12 to FR-14, FR-20 to FR-22, FR-26, FR-27
fp_analytics	FR-28
storage	FR-30, FR-31
timer_view	FR-01 to FR-11
room_canvas	FR-15 to FR-18

Module	Requirements primarily satisfied
wardrobe_dialog	FR-19 to FR-21
task_view	FR-22 to FR-27
charts_canvas	FR-28
calendar_view	FR-29
main	FR-31 to FR-33, controller wiring
theme	FR-32, NFR-07, NFR-11

5. Phase 2: System Architecture and Design

Phase 2 turns the requirements into a structure. The goal is to decide where each responsibility lives, how data moves, and which rules every module must obey, so that the later phases can be built and tested independently.

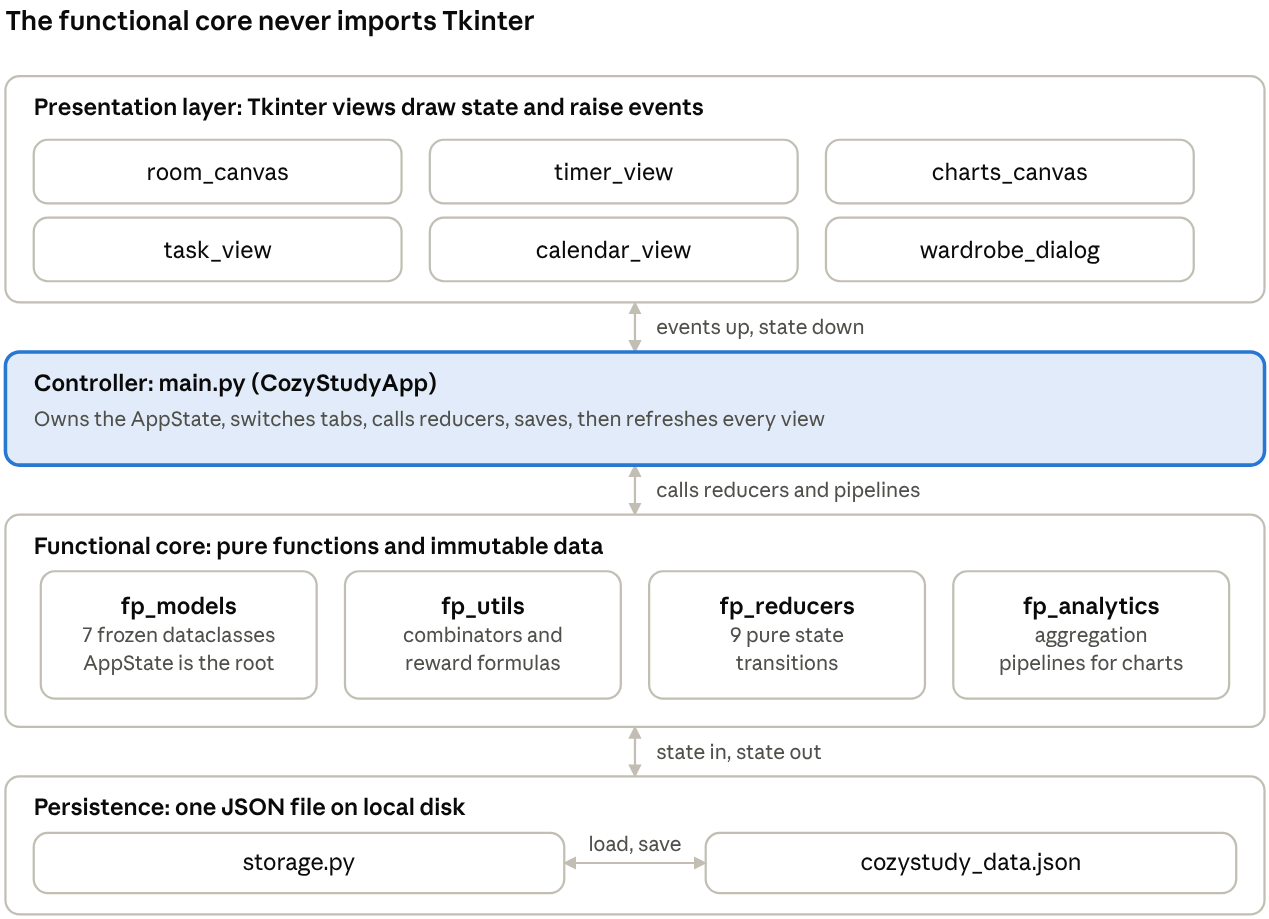
5.1 Architectural drivers

Four drivers shaped the design more than any others.

Driver	Consequence for the design
No third-party packages (NFR-02)	Tkinter and Canvas are the only GUI tools; charts and the room are drawn by hand
Business rules must be testable without a display (NFR-04)	Rules live in modules that never import tkinter
Many screens show the same data (coins, tasks, sessions)	A single source of truth, broadcast to every view
The project is also a demonstration of functional programming (O12)	Immutable models, pure reducers, pipelines, no in-place mutation

5.2 Chosen architecture: functional core, imperative shell

The design follows the well-known "functional core, imperative shell" idea. The **core** is a set of modules made only of immutable data and pure functions: given the same input they always return the same output and they change nothing outside themselves. The **shell** is everything that has side effects: drawing windows, reading the clock, playing sounds, showing message boxes and writing files.



CozyStudy layers · 4 layers, controller highlighted

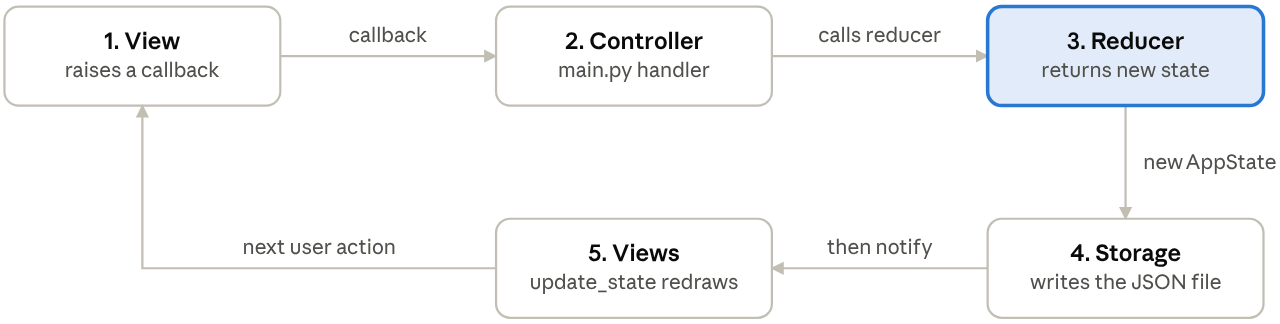
The figure shows the four layers. Arrows point both ways between layers because information travels down as events and calls and back up as new state, but the *rule* is strict about who may reach whom.

Layer	Modules	Allowed to do	Not allowed to do
Presentation	room_canvas, timer_view, charts_canvas, task_view, calendar_view, wardrobe_dialog, theme	Draw widgets and canvases; read the current state; call the callbacks they were given	Modify state directly; write files; call reducers directly
Controller	main	Hold the current AppState; call reducers; save; push state to views; show message boxes	Draw view contents itself (except the settings screen)
Functional core	fp_models, fp_utils, fp_reducers, fp_analytics	Compute and return new values	Import tkinter; touch the disk; read hidden global state (apart from the clock, see 5.9)
Persistence	storage, cozystudy_data.json	Convert state to and from dictionaries; read and write the JSON file; supply seed data	Contain business rules

5.3 One-way data flow

Every change in the application follows the same five-step loop, shown below. A view never edits data. It calls a function that the controller handed it. The controller passes the current state and the details of the action to a reducer. The reducer returns a brand-new state. The controller saves that state, and then tells every view to redraw from it.

State changes only through a pure reducer



one-way data flow · 5 steps, 1 loop

Three concrete scenarios illustrate the loop.

Scenario A: completing a task.

Step	Where	What happens
1	task_view	The user presses the Done button; the lambda calls on_complete_task(task_id)
2	main._on_complete_task	Calls fp_reducers.complete_task(self.state, task_id)
3	fp_reducers.complete_task	Finds the task, computes the bonus, returns a new state with the task marked completed and coins increased, plus the bonus value
4	storage.save_state	The new state is written to cozystudy_data.json
5	main._sync_state_to_all_views	The coin pill text is updated and every view receives update_state(new_state); then a message box announces the reward

Scenario B: finishing a focus session.

Step	Where	What happens
1	timer_view._timer_tick_loop	Remaining seconds reach zero and _handle_timer_completed runs

Step	Where	What happens
2	timer_view	Computes the integrity rate and the reward, finds the selected task, builds an immutable FocusSession
3	main._on_session_complete	Calls fp_reducers.add_session(self.state, session)
4	fp_reducers.add_session	Prepends the session, adds the reward to coins, increments the linked task's completed cycles, advances the streak
5	main	Saves and syncs; the timer view then shows a summary message

Scenario C: unlocking a charm.

Step	Where	What happens
1	wardrobe_dialog._handle_unlock	Checks that the balance covers the cost, otherwise shows an information box
2	main._on_unlock_charm	Calls fp_reducers.unlock_charm
3	fp_reducers.unlock_charm	Returns a new state with coins reduced, the charm unlocked and equipped, plus a success flag
4	main	If successful, saves and syncs; the room canvas immediately draws the new charm
5	wardrobe_dialog	The dialog closes itself

5.4 Module dependency rules

Imports in the code form a strict hierarchy with no cycles. A module may import only from modules listed in its row.

Module	Imports from the project	Third-party
fp_models	none	none
fp_utils	none	none

Module	Imports from the project	Third-party
fp_reducers	fp_models, fp_utils	none
fp_analytics	fp_models, fp_utils	none
storage	fp_models	none
theme	none	none
room_canvas	fp_models, theme	none
timer_view	fp_models, fp_utils, theme	none
charts_canvas	fp_models, fp_analytics, theme	none
task_view	fp_models, fp_utils, theme	none
calendar_view	fp_models, theme	none
wardrobe_dialog	fp_models, fp_utils, theme	none
main	storage, fp_models, fp_reducers, theme, all six view modules	none

The fp_models and fp_utils modules sit at the bottom and depend on nothing. Because nothing in the core imports a view, the core could be reused unchanged with a different front end, such as a web server or a command-line interface.

5.5 Design patterns used

Pattern	Where it appears	Benefit
Immutable value object	Every dataclass in fp_models is frozen	State cannot be corrupted by accident; safe to share between views
Reducer (state transition function)	fp_reducers	One place for each rule; easy to unit test
Observer by callback	Views receive callables such as on_add_task and on_session_complete	Views stay ignorant of the controller's internals
Broadcast refresh	_sync_state_to_all_views calls update_state on each view	Views are always consistent with each other

Pattern	Where it appears	Benefit
Facade	storage exposes load_state and save_state to hide JSON details	The controller never sees dictionaries
Pipeline and fold	fp_analytics uses reduce, filter and map to produce chart data	Declarative aggregation, easy to extend
Higher-order functions and currying	fp_utils.pipe, compose, curry, update_in_tuple	Reducers stay short and readable
Strategy by data	THEMES dictionary selected by a string key	New themes need no code change
Game loop (timer-driven animation)	after(100) in room_canvas and after(1000) in timer_view	Animation and timekeeping inside Tkinter's single thread
Defensive default	dict_to_state falls back to seed data	Start-up never fails on bad data

5.6 Key design decisions and rationale

ID	Decision	Alternatives considered	Rationale
D1	Tkinter for the interface	Web page, PyQt, Pygame	Ships with Python; nothing to install; adequate for forms and canvas drawings
D2	Frozen dataclasses	Plain classes, dictionaries, named tuples	Immutability enforced by the language; readable field access; free equality
D3	Tuples for collections in state	Lists	Tuples cannot be appended to in place, which supports immutability
D4	Reducers return new state via dataclasses.replace	Methods that mutate	Pure, testable, easy to reason about

ID	Decision	Alternatives considered	Rationale
D5	One JSON file	SQLite, several files	Human-readable; trivial backup; data volume is tiny
D6	Whole-state broadcast to all views	Fine-grained change events	Simplicity; state is small; views rebuild cheaply
D7	Window focus as the distraction signal	Polling the active application, browser extension	Works with the standard library on every platform
D8	Sessions store a copy of the task title	Reference by id only	A session remains readable after its task is deleted (the saved data already contains such orphans)
D9	Timer state kept inside TimerView, not in AppState	Persisting a running timer	Keeps the persisted model simple; cost is that an unfinished session is lost on exit
D10	Theme as a dictionary of colour tokens	Hard-coded colours	One place to change the look; four themes at no extra logic
D11	Charts drawn manually on Canvas	Third-party plotting library	Respects the no-dependency rule and gives a consistent look

5.7 State model: what is persisted and what is transient

Kind	Examples	Stored in
Persistent domain state	Coins, tasks, sessions, charms, subjects, settings, stats, active task id	AppState, saved to JSON

Kind	Examples	Stored in
Transient timer state	is_running, remaining_seconds, active_focus_seconds, distracted_seconds, tab_switches, is_window_focused, session_start_time	Attributes of TimerView only
Transient view state	Current priority filter, current status filter, selected calendar date, viewed month, lamp on or off, pet message, animation step	Attributes of the respective view
Derived data	Aggregate integrity, seven-day trends, timeline blocks, task statistics	Computed on demand by fp_analytics; never stored

Keeping derived data out of storage prevents it from becoming stale. For example the integrity percentage shown in the analytics screen is recomputed from the stored sessions whenever the screen refreshes.

5.8 Timing and concurrency model

The application is single-threaded. Tkinter's event loop runs all callbacks one after another, so there are no locks and no race conditions in the application code. Time-based behaviour uses the widget method after, which schedules a function to run later on the same event loop.

Loop	Period	Owner	Work done each tick
Timer tick	1000 ms	timer_view._timer_tick_loop	If running, subtract one second; add it to focused or distracted time; redraw the ring; detect completion
Room animation	100 ms	room_canvas._start_animation_loop	Increase the animation step; expire the pet message after 40 steps; redraw the whole room

Loop	Period	Owner	Work done each tick
Window events	on demand	main._on_global_focus_in and _on_global_focus_out	Forward focus changes to the timer view

A consequence of using after(1000) is that the timer counts ticks, not wall-clock time. If the machine sleeps or the event loop is blocked by a modal dialog, the countdown does not catch up. This is a limitation noted in Chapter 10.

5.9 Purity and its honest limits

The core is *almost* pure. Two functions consult the system clock when no date is supplied: complete_task (to decide whether a task is on time) and the helper _compute_streak (to find today and yesterday). The default value for Task.created_at and UserStats.last_active_date also use the clock. These are deliberate conveniences, and the code lets callers inject a date for testing in complete_task and the Task methods. _compute_streak does not accept an injected date, so streak behaviour is harder to test than it should be.

5.10 Error-handling strategy

Situation	Handling
Data file missing	load_state creates default seed data and saves it
Data file unreadable or invalid JSON	Message printed to the console; default state used and saved
Data present but a field is malformed	dict_to_state catches the exception, prints it and returns defaults
Missing subjects or charms in file	Replaced by the seed subjects or charms
Missing tasks or sessions in file	Treated as empty
Save fails (disk full, permission)	save_state returns False after printing; the caller ignores the result, so the user is not told
Invalid task form input	Only an empty title is rejected with a message box
Unknown theme key	Falls back to the Sakura palette

5.11 Privacy and security considerations

All data stays on the machine. There are no network calls, no accounts and no telemetry. The JSON file is stored in plain text next to the program, so anyone with access to the folder can read or change it. Because the data are low-sensitivity (study logs), the project does not encrypt them. The application never executes content from the data file; it only reads it as JSON, so a tampered file can at worst cause the defensive fallback to run.

5.12 Cross-cutting concern: theming

Theme tokens are shared by every view through theme.THEMES. Each palette defines the same keys (for example bg, bg_card, accent, text_primary, border, success, warning, danger and room colours), so a view can switch palettes by looking up a different dictionary. Typography tokens (FONT_TITLE, FONT_SUBTITLE, FONT_BODY, FONT_BODY_BOLD, FONT_SMALL, FONT_TIMER and FONT_EMOJI_BIG) all use the Helvetica family so that sizes, rather than families, create hierarchy.

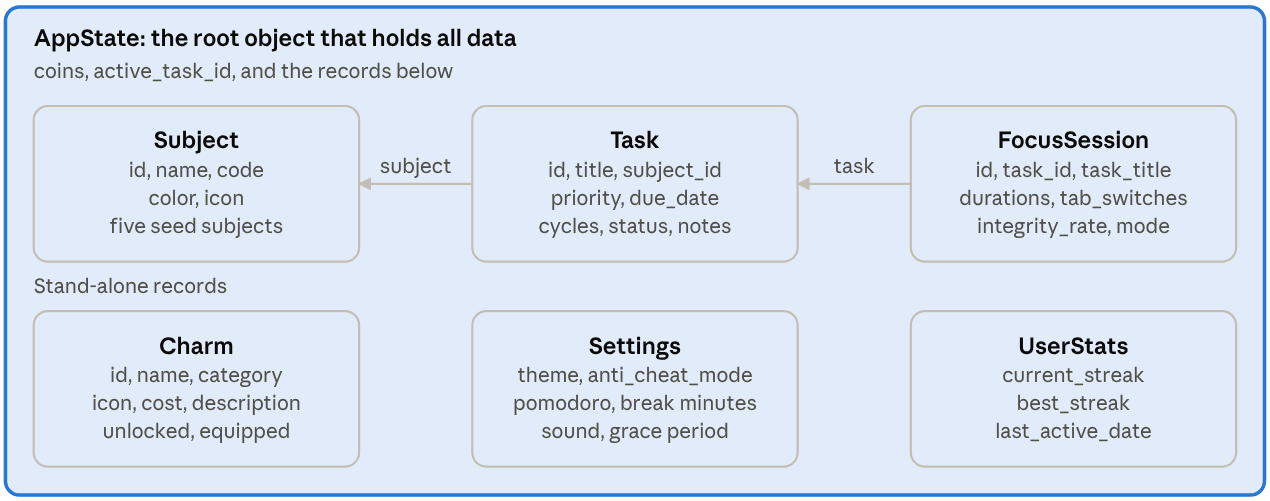
6. Phase 3: Back-End Design (Functional Core and Persistence)

CozyStudy has no server. In this report the *back end* means everything that holds data and enforces rules without drawing anything: the five modules fp_models, fp_utils, fp_reducers, fp_analytics and storage, together with the JSON data file. This chapter describes each in detail.

6.1 Domain model (fp_models.py)

Seven dataclasses declared with `@dataclass(frozen=True)` describe all information the program knows. Frozen dataclasses raise an exception if code tries to assign to a field after creation, so the only way to "change" a record is to build a new one, usually with `dataclasses.replace`.

AppState owns every record, and records link by id



domain model · 7 records, 2 links

Tasks refer to subjects by id, sessions refer to tasks and subjects by id, and the root AppState holds tuples of each record type. The links are by value (a string id), not by object reference, which keeps the model easy to serialise.

Subject

Field	Type	Meaning
id	str	Stable key, for example cs101
name	str	Full course name
code	str	Short course code, for example CS 201
color	str	Pastel colour hex code for the subject
icon	str	Emoji used on task cards

Task

Field	Type	Default	Meaning
id	str	none	Unique key, generated as task_ plus the Unix time in seconds
title	str	none	What to do

Field	Type	Default	Meaning
subject_id	str	none	Link to a Subject
priority	str	none	high, medium or low
due_date	str	none	Text in YYYY-MM-DD form
estimated_cycles	int	none	Planned number of 25-minute cycles
completed_cycles	int	0	Cycles done so far
status	str	todo	todo, in_progress or completed
notes	str	empty	Free text
created_at	str	now (ISO)	Creation timestamp, generated by default_factory
completed_at	Optional[str]	None	Set when completed

Task has two small query methods. `is_overdue(reference_date_str)` returns true when the task is not completed, has a due date, and the due date is earlier than the reference date. `is_due_today(reference_date_str)` is true when the task is not completed and the due date equals the reference date. Both compare ISO date strings as text, which is correct only because the format YYYY-MM-DD sorts chronologically.

Charm

Field	Type	Default	Meaning
id	str	none	Key such as charm_matcha
name	str	none	Display name
category	str	none	desk, wall, pet or floor
icon	str	none	Emoji icon
cost	int	none	Price in Star Charms
unlocked	bool	False	Whether the user owns it
equipped	bool	False	Whether it is drawn in the room
description	str	empty	One-line flavour text

FocusSession

Field	Type	Default	Meaning
id	str	none	sess_ plus Unix time
task_id	Optional[str]	none	Linked task, or None for general study
task_title	str	none	Copy of the title at the time of the session
subject_id	str	none	Subject of the session
start_time	str	none	ISO timestamp of the first start
duration_seconds	int	none	Active plus distracted seconds
active_focus_seconds	int	none	Seconds with the window focused
distracted_seconds	int	none	Seconds with the window unfocused
tab_switches	int	none	Number of focus losses
integrity_rate	int	none	0 to 100
mode	str	honest	honest or strict
completed	bool	True	Always true for stored sessions
charms_earned	int	0	Reward credited

Settings and UserStats

Record	Field	Default
Settings	theme	sakura
Settings	anti_cheat_mode	honest
Settings	pomodoro_minutes	25
Settings	short_break_minutes	5
Settings	long_break_minutes	15
Settings	sound_enabled	True
Settings	grace_period_seconds	5 (declared, not used anywhere)

Record	Field	Default
UserStats	current_streak	1
UserStats	best_streak	1
UserStats	last_active_date	today's date

AppState

Field	Type	Meaning
coins	int	Star Charm balance
subjects	Tuple[Subject, ...]	Course list
charms	Tuple[Charm, ...]	Charm catalogue with ownership
tasks	Tuple[Task, ...]	All tasks, newest first when added
sessions	Tuple[FocusSession, ...]	Completed sessions, newest first
active_task_id	Optional[str]	Task chosen with Focus Now
settings	Settings	Preferences
stats	UserStats	Streak figures

6.2 Functional utilities (fp_utils.py)

The module offers thirteen small functions. The first group are generic combinators; the second group are domain formulas.

Function	Kind	Behaviour
pipe(value, *functions)	Combinator	Applies functions left to right: pipe(x, f, g) equals g(f(x))
compose(*functions)	Combinator	Right to left composition: compose(f, g)(x) equals f(g(x))

Function	Kind	Behaviour
curry(fn)	Combinator	Turns a function of N arguments into a chain of one-argument calls, using the function's declared argument count
map_tuple(fn, items)	Higher-order	Like map, but returns a tuple
filter_tuple(predicate, items)	Higher-order	Like filter, but returns a tuple
reduce_iterable(fn, items, initial)	Higher-order	A fold over any iterable
find_first(predicate, items, default)	Query	First matching item or the default
update_in_tuple(items, predicate, transform)	Immutable update	New tuple in which matching items are transformed
remove_from_tuple(items, predicate)	Immutable update	New tuple without matching items
clamp(low, high, value)	Math	Restricts a number to a range
calculate_focus_integrity(active, total)	Domain formula	Percentage of time focused
format_seconds_to_min_sec(seconds)	Domain formula	MM:SS text, never negative
calculate_charm_reward(integrity, seconds, strict)	Domain formula	Star Charms for a session

Focus Integrity formula

Integrity is the share of elapsed session time during which the window was focused:

integrity = round(clamp(0, 100, active_focus_seconds / total_elapsed_seconds * 100))

If the elapsed time is zero or negative the function returns 100, so an empty session is never penalised.

Active seconds	Total seconds	Result	Comment
1500	1500	100	Perfect focus
1420	1500	95	First seeded session (94.67 rounds up)
1125	1500	75	375 seconds distracted
0	0	100	Empty session guard
1500	1000	100	Over-range ratio is clamped

Star Charm reward formula

reward = max(2, round(base x multiplier)), where base = max(1, duration_seconds // 100), the multiplier depends on integrity, and strict mode adds 0.2.

Integrity band	Multiplier (honest)	Multiplier (strict)
90 to 100	1.3	1.5
80 to 89	1.0	1.2
60 to 79	0.5	0.7
Below 60	0.2	0.4

The table below gives the rewards for a standard 1500-second session. The values were obtained by running the function.

Integrity	Honest reward	Strict reward
100, 95 or 90	20	22
89 or 80	15	18
79 or 60	8	10
59 or 0	3	6

Two observations are worth recording. First, the strict-mode bonus is small (about 10 percent), so strict mode is chosen for discipline, not for profit. Second, Python's round function rounds halves to the nearest even number, which is why 7.5 becomes 8 and 22.5

becomes 22 and 10.5 becomes 10. The sequence is correct but mildly surprising and is worth a comment in the code. Finally, the README says charms are earned "when you finish sessions with at least 80 percent integrity", but the code also pays 8 or 3 charms below that threshold. The behaviour in the code is more forgiving than the documentation.

6.3 Pure reducers (fp_reducers.py)

Each reducer receives an AppState and returns a new AppState (sometimes with an extra result). None of them touches the disk or the screen.

Reducer	Inputs	Output	Business rule
add_task	state, task	state	New task goes to the front of the tuple
update_task	state, task_id, field updates	state	Replace matching task with an updated copy; defined but not called by any view
delete_task	state, task_id	state	Removes the task; clears active_task_id if it pointed at it
complete_task	state, task_id, optional today	(state, bonus)	Marks completed and stamps completed_at; coins rise by the bonus
unlock_charm	state, charm_id	(state, success)	Needs enough coins; deducts cost; sets unlocked and equipped true
toggle_equip_charm	state, charm_id	state	Flips equipped, only for unlocked charms
add_session	state, session	state	Prepends session; adds reward; adds a cycle to the task; moves todo to in_progress; updates streak
set_active_task	state, task_id	state	Stores the Focus Now selection
update_settings	state, field updates	state	Replaces the settings record

Task completion bonus

bonus = 20, plus 15 if due_date is on or after today, plus 10 if priority is high. The bonus therefore ranges from 20 (late, not high) to 45 (on time and high). The date comparison is a text comparison of YYYY-MM-DD strings.

Streak algorithm

The private helper _compute_streak follows this logic:

1. If the stored last active date equals today, return the stats unchanged.
2. Otherwise compute yesterday's date.
3. If the last active date equals yesterday, the current streak increases by one; any other value resets it to one.
4. The best streak becomes the larger of the old best and the new current streak.
5. The last active date becomes today.

The algorithm is standard. Its practical reliability is undermined by a persistence defect described in Section 10.1.

6.4 Analytics pipelines (fp_analytics.py)

Analytics functions turn the stored sessions and tasks into figures that the charts display. They are declared as pure selectors and pipelines.

Function	Input	Output	Technique
get_today_sessions	sessions, optional date	tuple of sessions starting on the date	filter_tuple with a prefix match on the ISO start time
get_sessions_in_days	sessions, days	sessions newer than the cutoff	filter_tuple; text comparison of ISO timestamps
compute_aggregate_integrity	sessions	dictionary of totals and overall integrity	fold with functools.reduce over an accumulator dictionary
compute_daily_timeline	sessions, optional date	list of block dictionaries for one day	filter, sort by start time, map to blocks with colour and label

Function	Input	Output	Technique
compute_7_day_trends	sessions	list of seven daily dictionaries	group by date with defaultdict, then map over the last seven dates
compute_task_statistics	tasks	counts and completion rate	several filter_tuple counts

The timeline colour rule is: integrity 90 or above is green (Flawless Focus), 75 to 89 is amber (Mild Distraction), and below 75 is red (Tab Switching Alert). Two functions, get_today_sessions and get_sessions_in_days, and the task statistics function are not called by any view in the supplied code; they are available for future use.

Aggregate figures for the seeded sessions

Running compute_aggregate_integrity over the six seed sessions gives the following figures, which are the numbers a new user sees on the analytics screen.

Measure	Value
Sessions counted	6
Active focus seconds	8,500 (141.7 minutes)
Distracted seconds	800 (13.3 minutes)
Total scheduled seconds	9,300
Tab switches	16
Overall integrity	91 percent

6.5 Seed data (storage.py)

On the first run, or after a reset, the application starts with demonstration data so that every screen has something to show.

Subjects

Id	Name	Code	Icon
cs101	Data Structures and Algorithms	CS 201	laptop
math	Linear Algebra and Calculus	MATH 150	ruler
bio	Molecular Biology	BIO 110	DNA
lit	World Literature and Essay	LIT 102	book
art	Digital Art and Design	ART 210	palette

Charm catalogue

Charm	Category	Cost	Owned at start
Steaming Matcha Latte	desk	0	Yes
Pastel Desk Lamp	desk	0	Yes
Sleeping Calico Cat (Mochi)	pet	0	Yes
Baby Succulent Pot	desk	30	Yes
Twinkling Fairy Lights	wall	50	Yes
Vintage Lo-Fi Cassette Radio	desk	75	No
Fluffy Mochi Bunny	pet	90	No
Amethyst Focus Crystal	desk	120	No
Polaroid Memory Board	wall	80	No
Giant Monstera Deliciosa	floor	100	No
Velvet Cloud Beanbag	floor	110	No
Floating Moon Globe	desk	160	No

The starting balance is 85 Star Charms. The seven locked charms cost 735 in total, so a user needs to earn about 650 more than the starting balance to own everything. At roughly 15 to 20 charms per good session plus task bonuses of 20 to 45, that is a few

dozen sessions: a sensible long-term goal. The cheapest locked charm, the radio at 75, is affordable immediately.

Tasks and sessions

Five tasks (one already completed) and six past sessions are generated relative to the current date, so the demo always looks fresh. Their due dates fall between one and six days ahead. Starting streak figures are 3 current and 5 best.

Task	Subject	Priority	Cycles done / planned	Status
Implement Red-Black Tree Balanced Insert	cs101	high	1 / 3	in_progress
Linear Algebra Problem Set 4 (Eigenvalues)	math	high	0 / 2	todo
Literature Comparative Synthesis Essay	lit	medium	2 / 4	in_progress
Review Cellular Respiration Diagram and Enzymes	bio	low	1 / 1	completed
Design Pastel Palette and Room Moodboard	art	medium	0 / 2	todo

The seed sessions record charms earned of 15, 12, 20, 8, 18 and 15. Applying the real reward formula to the same integrity and duration values gives 20, 15, 20, 8, 23 and 20, so only two of the six seed rewards agree with the formula. The seed values are illustrative and do not matter for behaviour, but anyone comparing the demo to live results may notice the difference.

6.6 Persistence (storage.py)

File location and format

The data file cozystudy_data.json sits in the same folder as the program. It is written with two-space indentation. Non-ASCII characters such as emoji are written as \u escape sequences, which is valid JSON and keeps the file ASCII-safe.

Top-level JSON schema

Key	JSON type	Content
coins	number	Balance
active_task_id	string or null	Focus Now selection
subjects	array of objects	Subject fields
charms	array of objects	Charm fields
tasks	array of objects	Task fields
sessions	array of objects	FocusSession fields
settings	object	Settings fields
stats	object	UserStats fields

Serialisation and deserialisation

`state_to_dict` converts each record with its **dict** attribute, which is possible because the dataclasses do not use slots. `dict_to_state` rebuilds each record with keyword expansion, for example `Task(**t)`. For the two nested singleton records it filters keys using `hasattr(Class, key)` so that unknown or obsolete keys in an old file are ignored instead of raising an error. That filter has an unintended side effect for fields declared with default_factory; see Section 10.1.

Load and save behaviour

Function	Behaviour
load_state	If the file exists, parse it and convert; on any exception print the error and fall through; otherwise build default state, save it and return it
save_state	Convert to dictionary and write the file; return True on success, False on failure (error printed)
get_default_state	Assemble seed subjects, charms, tasks, sessions with 85 coins and streak figures 3 and 5

What the supplied data file shows

The cozystudy_data.json supplied with the project is a real snapshot of use, not seed data, and it is informative.

Observation	Evidence	Meaning
Theme is midnight	settings.theme	The user switched to the dark theme
Balance is 10	coins = 10	Starting 85 minus the 75 paid for the radio
Radio is unlocked and equipped	charm_radio flags	The user bought the first affordable charm
Six charms are unlocked	Count of unlocked flags	Five free or seeded plus the radio
Task task-1 is missing	tasks list has four entries	The user deleted it
Two sessions still reference task-1	sess-1 and sess-4	Sessions survive task deletion because they copy the title (decision D8)
All due dates are in September 2026	due_date values	Three tasks are now overdue relative to 16 October 2026
last_active_date is 2026-10-16 although sessions end on 13 September	stats	Evidence of the streak persistence defect

Applying compute_task_statistics to this file yields 4 tasks, 1 completed, 3 overdue, 0 due today, 1 high-priority open and a 25 percent completion rate. Applying compute_7_day_trends yields seven days of zero focus minutes, because every stored session is more than a week old.

6.7 Complexity and scalability

Every reducer and analytics function is a linear scan over a tuple, so the cost per operation is proportional to the number of records, written O(n). With hundreds of tasks and thousands of sessions this is still instantaneous. The main limitation on scale is not

computation but that the entire state is rewritten to disk after every change. For a student's lifetime of study (a few thousand sessions, each about 300 bytes) the file would remain below a megabyte, so the simple design is justified.

6.8 Back-end quality attributes

Attribute	Assessment
Correctness	Formulas are simple and verified by unit tests; two defects found (Chapter 10)
Testability	Excellent: no GUI import, no I/O in reducers
Readability	High: short functions with docstrings
Extensibility	High: a new rule means a new reducer; a new chart means a new pipeline
Robustness	Good on load; weak on save (errors ignored, no atomic write)
Efficiency	More than adequate for the data size

7. Phase 4: Front-End Design (Tkinter Views)

The front end is everything the user sees and touches. It consists of the main window shell, six full-screen tabs, two pop-up dialogs, a theme system and a set of hand-drawn Canvas illustrations. This chapter documents the design language, each screen, the interaction rules and the drawing techniques.

7.1 Design goals and visual language

The interface is meant to feel like a calm, personal corner rather than a productivity dashboard. Six rules recur throughout the code.

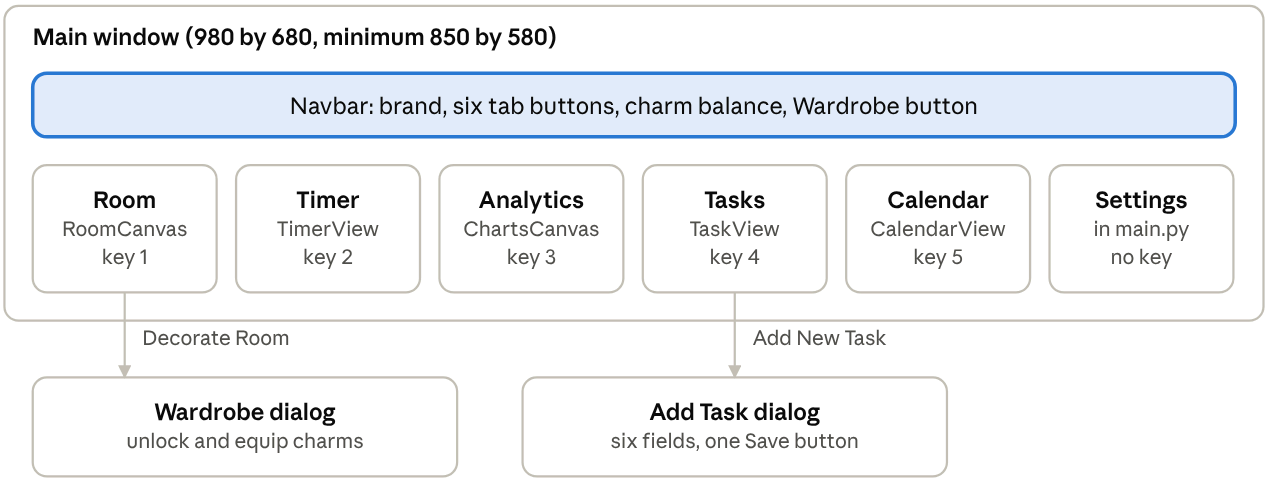
Rule	How it shows
Soft cards on a tinted background	Every panel is a white or palette-card frame with a one-pixel pastel border on a slightly tinted page background

Rule	How it shows
One accent colour per theme	Buttons, active tabs and highlights use the palette's accent; success, warning and danger colours carry status
Emoji as icons	Tab labels, buttons and charm icons use emoji, so no image files are needed
Rounded information, not decoration	Pills and badges (streak, charm balance, priority, integrity) summarise state in a few words
Immediate feedback	Status pills change colour at once; message boxes confirm rewards
Small fixed type scale	Seven font tokens, one family

7.2 Window shell and navigation

The application is a single main window of 980 by 680 pixels, resizable down to 850 by 580. The window contains a navigation bar across the top and a content area below. Only one of the six views is visible at a time; switching a tab hides the others with pack_forget, shows the chosen one, and asks it to refresh from the latest state.

One window holds six tabs and two dialogs



screen map · 6 tabs, 2 dialogs

Navigation bar contents

Region	Content	Behaviour
Left	Brand label "CozyStudy" in the accent colour	Static
Centre	Six tab buttons: Room, Focus Timer, Analytics, Tasks, Calendar, Settings	The active tab uses the palette's tab_active colours; others use muted text
Right	Charm balance pill and a Wardrobe button	Both open the Wardrobe dialog; the pill text updates after every state change

Global interaction rules

Input	Effect
Keys 1, 2, 3, 4, 5	Switch to Room, Timer, Analytics, Tasks, Calendar
Window gains focus	Timer is told the user is back
Window loses focus	Timer records a tab switch and, in strict mode, pauses
Click on the cat	A random encouraging message appears above it for four seconds
Click on the lamp	The light cone toggles on and off

7.3 Theme system

Each theme is a dictionary of twenty-two keys. The four palettes follow.

Theme	Mood	Page background	Accent	Primary text	Room wall
Sakura Blossom	Soft blush pink and cream	#FFF5F7	#E79AAB	#4A3B40	#FDE8EC
Matcha Study	Botanical greens and oat	#F4F7F2	#7A9A60	#323E2A	#EAF0E5
Lavender Haze	Misty lilac	#F8F6FC	#9A86C4	#3B364A	#EEE8F8
Midnight Lo-Fi	Dark navy with amber warmth	#1E1E2E	#CBA6F7	#CDD6F4	#26243D

Token groups

Group	Keys
Surfaces	bg, bg_card, bg_card_secondary
Accent	accent, accent_hover, accent_light
Text	text_primary, text_secondary
Lines	border
Room	room_wall, room_floor, room_baseboard, room_window_sky_day, room_window_sky_night
Status	success, warning, danger
Currency	charm_coin
Tabs	tab_active_bg, tab_active_fg

A few colours are not tokenised and are written directly in the views: the white of speech bubbles, the pale red (#FEE2E2) used for the "Tabbed Away" pill and the high-priority badge, and many illustration colours in the room. In the Midnight theme these hard-coded light colours appear as bright patches on a dark page; this is a minor visual inconsistency.

7.4 Typography and iconography

Token	Specification	Used for
FONT_TITLE	Helvetica 18 bold	Screen titles
FONT_SUBTITLE	Helvetica 13 bold	Card headings, main buttons
FONT_BODY	Helvetica 11	Normal text
FONT_BODY_BOLD	Helvetica 11 bold	Labels, tab buttons
FONT_SMALL	Helvetica 9	Captions, small buttons
FONT_TIMER	Helvetica 40 bold	Countdown digits
FONT_EMOJI_BIG	Helvetica 28	Large emoji (declared)

7.5 Screen specifications

Each of the six screens and the two dialogs is specified below: its purpose, layout, controls and behaviour.

7.5.1 Screen 1: Room (RoomCanvas)

Purpose. A reward and motivation space. It shows what the student has earned.

Layout. A header strip with the title, a streak badge (flame plus number of days) and a Decorate Room button that shows the current balance. Below it, a single Canvas fills the rest of the window and is redrawn whenever it is resized.

Scene geometry. All positions derive from the canvas width w and height h, so the scene scales with the window.

Element	Position and size
Floor line	y = 0.62 h; wall above, floor below; baseboard 14 px tall; diagonal floor planks every 45 px
Rug	Ellipse centred at (0.48 w, floor_y + 0.48 (h - floor_y)); radii min(0.35 w, 220) and min(0.38 (h - floor_y), 70); inner dashed ring at 85 percent
Window	Width min(160, 0.28 w), height min(180, 0.65 floor_y), at x = 0.70 w, y = 0.15 floor_y; cross bars and a sill
Bookshelf	At x = 0.08 w, y = 0.22 floor_y; width min(110, 0.2 w); two shelves, five coloured books, one plant
Desk	Width min(240, 0.42 w) at x = 0.45 w, y = 0.72 floor_y; two legs, desk mat, open laptop with three coloured code lines
Chair	Beside the desk, with a cushion in the accent colour

Window behaviour. The window shows a pale-blue sky with a sun and a drifting cloud in the three light themes, and a dark sky with a crescent moon and five twinkling stars in the Midnight theme. The choice depends on the selected theme, not on the time of day (the README's wording suggests a clock-based change; the code does not read the clock here).

Charm placement

Charm	Drawn as	Anchor
Matcha latte	Green cup with handle and two steam strokes	desk_x + 0.8 desk_w, on the desk
Desk lamp	Pole, shade in accent colour, optional translucent light cone	desk_x + 18
Calico cat	Oval body with patches, head, ears, tail, floating z	Rug centre + (10, 8)
Mochi bunny	White oval, long ears, tiny face text	Rug centre - 80 px
Succulent	Emoji	desk_x + 0.12 desk_w
Radio	Brown box, dial, antenna, rising musical note	desk_x + 0.9 desk_w
Crystal	Emoji	desk_x + 0.04 desk_w
Polaroid board	String, two photo cards with clothespins	(0.32 w, 0.2 floor_y)
Monstera	Large emoji and a pot	(0.05 w, floor_y + 15)
Beanbag	Two nested ovals	(0.92 w, floor_y + 35)
Moon globe	Emoji with vertical bobbing	desk_x + 0.7 desk_w
Fairy lights	Seven bulbs on a drooping wire above the window	Window top edge

Animation catalogue

A counter named animation_step increases every 100 ms. Each animation is a simple function of that counter, so the whole scene is redrawn from scratch each frame.

Animation	Formula (step = animation_step)	Visible effect
Steam rising	offset = (step mod 20) x 0.8	Two strokes drift up and loop
Cat breathing	1.5 x sin(0.25 step)	Body height changes by up to 1.5 px
Tail wag	3 x sin(0.18 step)	Tail tip sways

Animation	Formula (step = animation_step)	Visible effect
Sleep particle	position moves by (step mod 30) x (0.6, 1.2)	A z floats up and left
Cloud	x offset (step mod 120) x 0.1	Slow drift, then restart
Stars (Midnight)	brightness from sin(0.4 (step + 2 i))	Stars alternate white and grey
Fairy bulbs	Colour changes when (step + i) mod 3 is 0	Gentle flicker
Radio note	y rises (step mod 20) x 0.5	Note floats up
Moon globe	2 x sin(0.2 step)	Floats up and down
Pet message	Cleared after 40 steps	Message bubble disappears after four seconds

Click targets are circles: radius 45 pixels around the rug centre for the cat and 30 pixels around the lamp shade for the lamp. The cat is drawn slightly to the right and below the rug centre, so clicks on the left edge of the cat may feel just outside the target.

7.5.2 Screen 2: Focus Timer (TimerView)

Purpose. The core of the application: run a session and measure honest focus.

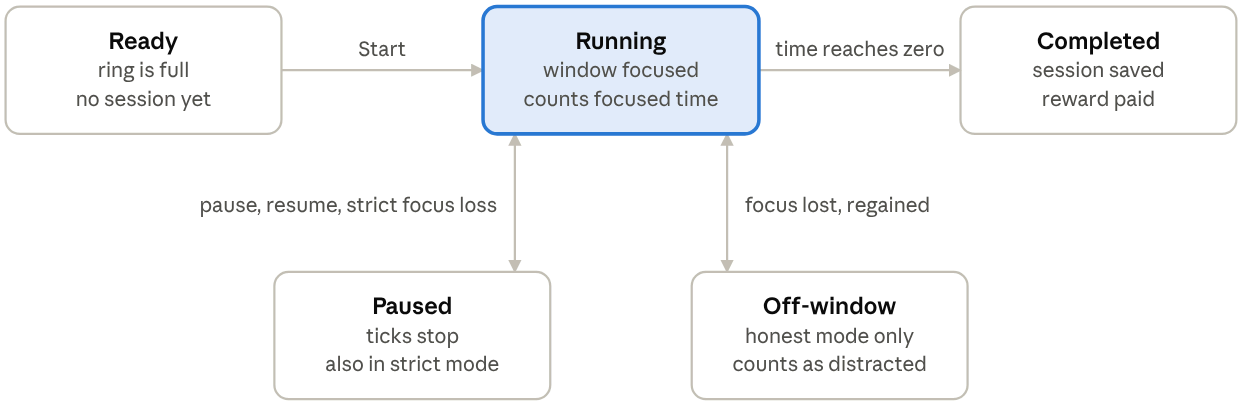
Layout, from top to bottom.

1. A top bar with three mode buttons (Pomodoro 25 minutes, Short Break 5 minutes, Long Break 15 minutes) on the left and the anti-cheat mode selector on the right.
2. A target-task row with a drop-down listing "General Study" and every unfinished task, labelled with its priority.
3. A 280 by 280 canvas holding a circular progress ring (radius 112 pixels, 12 pixel stroke) with the MM:SS digits at its centre and the caption FOCUS TIME or BREAK TIME.
4. A status pill: Present and Focused, Ready to Focus, or Tabbed Away with the seconds off-window.
5. An integrity card with a percentage, a thin bar and the line "N tab/window switches | N s off-focus".
6. Start, Pause or Resume and Reset buttons.

Ring colour. The arc is green while focused, red while the window is unfocused and the timer runs, and the accent colour in break modes.

State machine.

Honest mode keeps counting off-window while strict mode pauses



Reset or a mode switch returns any state to Ready

timer states · 5 states, anti-cheat modes

Transition table

From	Event	To	Effect
Ready	Start pressed	Running	Records the session start time
Running	Pause pressed	Paused	Ticks stop; button reads Resume Focus
Paused	Resume pressed	Running	Ticks continue
Running	Window loses focus (honest)	Off-window	Tab-switch counter increases; alert sound; red pill; seconds now count as distracted
Off-window	Window regains focus	Running	Pill returns to green
Running	Window loses focus (strict)	Paused	Tab-switch counter increases; warning dialog; button reads Resume (Paused by Anti-Cheat)

From	Event	To	Effect
Running or Off-window	Remaining time reaches zero	Completed	Integrity and reward computed; session reported to the controller; summary dialog
Any	Reset, or switching mode	Ready	Counters cleared; no session is recorded

What counts as time

Each second that passes while the timer runs in focus mode is added either to active focus seconds (window focused) or to distracted seconds (window unfocused). The sum is the session's duration. Pausing adds nothing. Breaks add nothing to either counter and never create a session.

Integrity meter colours

The live meter shows green at 85 percent or higher, amber from 70 to 84 and red below 70. The analytics timeline uses different thresholds (90 and 75), and the README quotes the same numbers as the timeline. A student may therefore see a green live meter at 86 percent and an amber timeline block for the same session. Aligning the two sets of thresholds is recommended in Chapter 12.

7.5.3 Screen 3: Analytics (ChartsCanvas)

The screen scrolls vertically inside a canvas with a scrollbar and contains four blocks.

Block	Content	Drawing method
KPI cards	Pure focus hours; overall integrity with tab-switch count; completed cycles; streak with best streak	Four equal cards built from labels
Donut chart	Focus versus off-tab minutes with central integrity percentage and a legend	Canvas ovals and arcs; 280 by 210 canvas; ring radius 70, stroke 26
Seven-day bars	Stacked bar per day: focus in green, off-tab in red, minute label above	Canvas rectangles; 340 by 210 canvas; bar width 26, spacing 42, baseline at y = 175, plot height 140; scale never below 60 minutes

Block	Content	Drawing method
Today's timeline	One row per session today: time, title, minutes focused and distracted, switches, and an integrity badge	Frames and labels; colour bar on the left

The donut is drawn by painting a full red ring first and then overlaying the green arc clockwise from twelve o'clock. If there is no data the ring is grey with the words No Data. When there are no sessions today the timeline shows a friendly empty message.

7.5.4 Screen 4: Tasks (TaskView)

Header. Title and an Add New Task button.

Filter bar. Five buttons: All Active, High, Medium, Low, Completed. The status filter is either pending or completed; the priority filter is all, high, medium or low.

Task card contents. A subject badge, a priority badge (coloured pale background with a status colour), a deadline label, the title, notes, a cycle counter (for example 1 / 3 cycles completed), and the buttons Focus Now, Done and a small delete cross.

Deadline condition	Label	Colour
Completed	Completed	Secondary text
Due date before today	Overdue with the date	Danger
Due date equals today	Due Today	Warning
Due later	Due followed by the date	Secondary text

Completed tasks show a struck-through title and no Focus Now or Done buttons. Deleting asks for confirmation.

Add Task dialog. A modal window of 450 by 420 pixels with fields for title, subject (drop-down), priority (drop-down, default medium), due date (text, default two days ahead), estimated Pomodoros (spin box from 1 to 10) and notes. Only an empty title is rejected. The due date is not validated, so a typing error such as 2026-13-45 is accepted and will sort and compare as text.

7.5.5 Screen 5: Calendar (CalendarView)

A header with previous and next month buttons and the month name. Below, a month grid with seven weekday headings and cells 46 by 46 pixels, and on the right a 280-pixel Day Inspector.

Cell state	Appearance
Ordinary day	Card background, border colour
Today	Secondary card background, accent border, bold number
Selected	Active-tab background, accent border
Day with a session	Small green dot
Day with an unfinished task deadline	Small red dot

The inspector shows the full date, a summary box (active focus minutes, number of sessions, tab switches), a list of that day's sessions (time, title, duration, integrity, switches) and, if any, a list of deadlines on that day. Month navigation never changes the selected day.

7.5.6 Screen 6: Settings

Built directly in main.py. It offers a theme selector, the anti-cheat mode, a checkbox for alert sounds, an Export Backup (JSON) button that opens a save dialog, and a Reset to Starter Demo Data button with a confirmation. Behaviour notes appear in Chapter 10: the settings screen does not update the timer's own mode selector, and a theme change only partly restyles screens that are already built.

7.5.7 Wardrobe dialog

A modal window of 640 by 520 pixels (minimum 550 by 450) with a header showing the balance, a one-line explanation, and a tabbed notebook with four categories: Desk Decor, Wall and Lights, Companions, and Floor and Plants. Each tab is a scrollable list of cards. A card shows the emoji, name, description and an action button. A locked charm shows Unlock with its cost, coloured accent when affordable and grey when not; pressing an unaffordable one explains how many charms are needed. An owned charm shows Equip or Equipped, plus an In Room tag when equipped. The dialog closes after every action, so changing several charms means reopening it each time.

7.6 Usability review

The table applies Nielsen's ten usability heuristics to the delivered interface.

Heuristic	Assessment
Visibility of system status	Strong: live pill, ring colour, integrity meter, balance pill
Match with the real world	Strong: Pomodoro, cards, cosy room metaphor
User control and freedom	Good: reset, pause, confirm on delete; but no undo for completing a task
Consistency	Good: shared cards and tokens; threshold mismatch noted in 7.5.2
Error prevention	Fair: modal dialogs and confirmations; free-text dates unvalidated
Recognition over recall	Good: icons and labels together
Flexibility and efficiency	Fair: keys 1 to 5 for tabs; no shortcut to start the timer
Aesthetic and minimalist design	Strong: calm layouts; some small 9-point text
Help users recover from errors	Fair: messages are friendly but few
Help and documentation	Weak: the README documents the application but there is no in-app help

Accessibility notes

Small fonts (9 point), emoji-only cues in places and colour-only status (green, amber, red) may be hard for some users. Labels such as Overdue and the percentage digits mitigate this. A larger-text option and a colour-blind-safe palette are suggested as future work.

7.7 Responsiveness

Only the Room canvas is fully fluid. The donut and bar charts use fixed canvas sizes, the calendar cells are fixed at 46 pixels, and the inspector is fixed at 280 pixels. At the minimum window width of 850 pixels all of these still fit; very wide windows simply leave extra background around them.

8. Phase 5: Implementation Details by Module

This chapter explains how the design was turned into code. It covers the development environment, the size of each module, the important algorithms and the techniques used, module by module. Short excerpts of the project's own code are quoted where they clarify the logic.

8.1 Development and runtime environment

Item	Detail
Language	Python 3 (the supplied tests were run for this report under Python 3.12.3)
GUI toolkit	Tkinter and ttk from the standard library
Operating systems	Written on macOS (the launcher mentions a Python 3.14 framework path and the sound code uses macOS system sounds); portable to Linux and Windows
Fonts	Helvetica, substituted by Tk when absent
External packages	None
Launch	python3 main.py, or sh run.sh
Entry point	main.main() creates CozyStudyApp and calls mainloop()
Path handling	Every module that needs the project folder resolves it from **file**, and main.py and storage.py insert it in sys.path so imports work from any current directory

8.2 Code metrics

The figures below were measured on the delivered files.

File	Lines	Layer
room_canvas.py	467	View

File	Lines	Layer
timer_view.py	466	View
main.py	413	Controller
task_view.py	370	View
charts_canvas.py	298	View
storage.py	293	Persistence
calendar_view.py	211	View
wardrobe_dialog.py	201	View
fp_reducers.py	162	Core
fp_analytics.py	144	Core
fp_utils.py	125	Core
test_app.py	117	Test
theme.py	114	Presentation
fp_models.py	103	Core
Total	**3,484**	

By layer, the views total 2,013 lines (58 percent), the functional core 534 lines (15 percent), the controller 413 lines (12 percent), persistence 293 lines (8 percent), theme 114 lines (3 percent) and tests 117 lines (3 percent). The front end is therefore about four times the size of the rules it displays, which is typical of a graphical application and reinforces the value of keeping the rules in a small, well-tested core.

8.3 main.py: the controller

Class. CozyStudyApp extends tk.Tk. It is the only object that holds the current AppState (self.state).

Start-up sequence (constructor).

1. Call storage.load_state() to obtain the initial immutable state.
2. Set the window title, size (980 by 680) and minimum size (850 by 580).
3. Look up the theme dictionary from state.settings.theme, defaulting to Sakura.

4. Build the shell: navigation bar, content area and the six views.
5. Bind global events and show the Room tab.

Shell construction. The navigation bar is built from a list of six (id, label) pairs. Each tab button carries a lambda that calls switch_tab with its id. The six views are created once and stored in a dictionary keyed by tab id; switching tabs only hides and shows them.

Callback methods. Each callback follows the same four-line recipe: call a reducer, save, synchronise the views and optionally show a message.

Callback	Reducer called	Extra behaviour
_on_session_complete	add_session	None
_on_add_task	add_task (after building Task from a dictionary)	None
_on_complete_task	complete_task	Message box with the bonus
_on_delete_task	delete_task	None
_on_focus_task	set_active_task	Tells the timer view and switches to the Timer tab
_on_unlock_charm	unlock_charm	Message box on success
_on_toggle_equip_charm	toggle_equip_charm	None
_on_theme_change	update_settings	Re-colours the window and navbar
_on_cheat_mode_change	update_settings	None
_on_sound_toggle	update_settings	Saves but does not synchronise

The recipe in code form:

```
def _on_complete_task(self, task_id: str):
    self.state, coins_earned = fp_reducers.complete_task(self.state, task_id)
    storage.save_state(self.state)
    self._sync_state_to_all_views()
    messagebox.showinfo("Task Complete", f"Great job! You earned +
{coins_earned} Star Charms!")
```

The single assignment to self.state is the only place a view-visible state change occurs; everything else is a function call that returns a value.

Synchronisation. _sync_state_to_all_views updates the charm balance pill and then calls update_state on every view that has such a method. Because the room, timer, analytics, task and calendar views each re-read what they need, no view can show stale data after an action.

Settings view. Built inline with two ttk.Combobox controls, one Checkbutton and two buttons. Backup uses filedialog.asksaveasfilename and writes the dictionary produced by storage.state_to_dict. Reset asks for confirmation and replaces the state with get_default_state.

8.4 timer_view.py: the anti-cheat engine

Instance attributes hold the whole timer state: is_running, timer_mode, total_duration_seconds, remaining_seconds, active_focus_seconds, distracted_seconds, tab_switches, is_window_focused and session_start_time.

The tick loop is the heart of the engine. It reschedules itself every second with after(1000):

```
def _timer_tick_loop(self):
    if self.is_running and self.remaining_seconds > 0:
        self.remaining_seconds -= 1
        if self.timer_mode == 'focus':
            if self.is_window_focused:
                self.active_focus_seconds += 1
            else:
                self.distracted_seconds += 1
        if self.remaining_seconds <= 0:
            self._handle_timer_completed()
        self._draw_timer_ring()
    self.after(1000, self._timer_tick_loop)
```

The loop is idempotent when paused: nothing changes unless the timer is running. A running timer in honest mode therefore keeps ticking when the window is unfocused and merely changes which counter receives the second.

Focus detection. The controller binds the Tk events FocusIn and FocusOut on the root window and forwards them, after checking that the event's widget is the root, to on_window_focus_in and on_window_focus_out. The latter increments tab_switches only

if the timer is running in focus mode, plays an alert, and in strict mode stops the timer and shows a warning.

Completion. _handle_timer_completed computes integrity, calls calculate_charm_reward, identifies the selected task, constructs a FocusSession and hands it to the controller. The task is identified by looking for each unfinished task's title inside the drop-down text. Because both the priority prefix and the title appear in the text, the match works, but two tasks with titles that contain each other would be ambiguous.

Sounds. If the macOS sound files exist, afplay is launched as a background process (Tink for an alert, Glass for completion); otherwise the Tk bell is used. Sound can be turned off in Settings.

Drawing. _draw_timer_ring clears the 280 by 280 canvas and repaints the track, the progress arc (extent minus 360 times the elapsed fraction, starting at the top), the digits, the caption, the integrity label and the integrity bar.

8.5 room_canvas.py: the diorama

redraw follows the painter's algorithm: back-to-front drawing so later items cover earlier ones.

Order	Item
1	Wall rectangle, floor rectangle, baseboard
2	Floor planks (diagonal lines every 45 px)
3	Rug (outer oval and dashed inner ring)
4	Window with sky, sun or moon, cloud or stars, frame, sill, fairy lights
5	Bookshelf
6	Chair, desk, legs, mat, laptop
7	Equipped charms in a fixed order (matcha, lamp, cat, bunny, succulent, radio, crystal, polaroids, monstera, beanbag, globe)
8	Pet message bubble if present

A helper _adjust_color adds or subtracts a number from each RGB channel and clamps to the range 0 to 255; the planks are drawn 15 levels darker than the floor. A second helper, is_charm_equipped, scans the charm tuple. The draw call count is a few hundred Canvas items per frame, which Tkinter handles easily at ten frames per second.

The animation loop does three things: increments the counter, clears an expired pet message, and redraws. It is started in the constructor and never stopped.

8.6 charts_canvas.py: analytics drawing

The module offers four render functions that are called from redraw_all.

Function	Key technique
_render_kpis	Destroys previous card widgets and rebuilds four cards from the aggregate dictionary
_render_donut_chart	create_oval for the track; create_arc with style ARC and negative extent for clockwise slices; text and rectangles for the legend
_render_trend_bar_chart	Computes the largest daily total (at least 60) to scale bars; draws distracted portion above focus portion
_render_timeline	Rebuilds one row per session from compute_daily_timeline output

The central percentage in the donut is the aggregate integrity, while the green and red proportions come from focus minutes against focus plus distracted minutes. For stored sessions these coincide because duration equals active plus distracted seconds.

8.7 task_view.py: tasks

The list is rebuilt from scratch on every refresh: destroy all card widgets, filter the tasks, render cards. Filtering is a plain loop with two guards (status and priority). A card is a frame containing rows for the badges, title, notes and actions. Button commands are bound with default-argument lambdas (command=lambda tid=task.id: ...), a standard idiom that prevents the loop variable from being shared between buttons.

8.8 calendar_view.py: month grid

Python's calendar.monthcalendar yields a list of weeks (each seven numbers, zero for padding). The view collects the dates that have sessions and the dates that have unfinished deadlines into two sets so that each cell can test membership in constant time. Cells are frames with bound click handlers on the frame and its labels, because a click on a child label would not otherwise reach the frame.

8.9 wardrobe_dialog.py: shop

The dialog is a Toplevel that is made modal with transient and grab_set. Each category tab builds a canvas with an inner frame, a scrollbar and one card per charm. Buttons are created conditionally: Unlock for locked charms (colour depends on affordability) and Equip or Equipped for owned ones. The handlers look the charm up with find_first, check the balance one more time, call the controller callback and close the dialog.

8.10 storage.py: persistence

The module has three seed builders for subjects, charms and tasks and sessions, one for the whole default state, two converters and two file functions. Seed tasks and sessions use datetime.now() with timedelta offsets so that their dates are always relative to the current day. The data file path is computed from the module's own location, which keeps the data with the program.

8.11 Functional-programming techniques in practice

Technique	Example in the code
Immutability	Every dataclass is frozen; reducers use dataclasses.replace
Immutable collections	update_in_tuple and remove_from_tuple return new tuples
Pure functions	calculate_focus_integrity, calculate_charm_reward, all reducers, all analytics
Higher-order functions	update_in_tuple takes a predicate and a transform; find_first takes a predicate
Folding	compute_aggregate_integrity uses functools.reduce with an accumulator

Technique	Example in the code
Closures	Lambdas capture task or charm ids in view buttons
Declarative selection	filter_tuple with small lambdas in analytics
Function returning function	curry and compose (available and tested)
Separation of effects	I/O only in storage and the views; none in the core

The README advertises pipelines built with pipe, compose, curry, map_tuple and reduce_iterable. In the delivered code pipe and map_tuple are imported by fp_analytics but never called, and compose, curry and reduce_iterable are not used outside the test file. The combinators exist and work, but the production code relies more on filter_tuple, update_in_tuple, find_first and functools.reduce.

8.12 Unused or partly used declarations

Item	Where	Note
update_task	fp_reducers	No view calls it; there is no edit-task screen
get_today_sessions, get_sessions_in_days	fp_analytics	Not called by any view
compute_task_statistics	fp_analytics	Not called; no task statistics panel
pipe, compose, curry, map_tuple, reduce_iterable	fp_utils	Tested but unused in production code
Settings.grace_period_seconds	fp_models	A five-second grace period for short focus loss is declared but never applied
Settings.short_break_minutes, long_break_minutes	fp_models	Used by the timer but there is no control to change them; the button captions always read 5 and 15 minutes
FONT_EMOJI_BIG	theme	Declared, never used
style = ttk.Style()	wardrobe_dialog	Created and unused

Item	Where	Note
Local variables w and h	charts_canvas	Assigned in two methods and not used

None of these affects behaviour. They suggest features that were planned (grace period, editable breaks, task statistics) and are good candidates for the next release (Chapter 12).

8.13 Coding conventions observed

Convention	Observation
Module docstrings	Present in every file and explaining purpose and features
Function docstrings	Present in the core; sparse in views
Type hints	Used consistently on core functions and many view signatures
Naming	snake_case for functions and variables, PascalCase for classes, leading underscore for internal methods
Layout	Four-space indentation; long constructor calls split one argument per line
Magic numbers	Many pixel offsets in the room drawing; a few thresholds (85, 70, 90, 75) repeated in different files
Error handling	Broad except clauses around file access, colour parsing and sound; acceptable at the edges

8.14 Platform notes

Platform	Notes
macOS	Fully supported; native sounds via afplay; run.sh falls back to the python.org framework path

Platform	Notes
Linux	Needs the Tk package installed (for example python3-tk on Debian and Ubuntu); sound falls back to the Tk bell; emoji need an emoji-capable font
Windows	Python from python.org includes Tk; use python main.py instead of run.sh; sound falls back to the bell

9. Phase 6: Testing and Verification

This chapter documents how CozyStudy is verified: the test strategy, the existing automated suite and its results, an honest assessment of coverage, and a complete set of additional designed test cases for the unit, GUI, system and non-functional levels. Where a result is reported as "run", it was obtained by executing the delivered code while preparing this report. Where a test is only "designed", it has not been run and the expected result is derived from reading the code.

9.1 Purpose and scope of testing

The purpose of testing is to give evidence that the application meets the requirements in Chapter 4: that the study timer measures focus correctly, that rewards and bonuses are computed according to the rules, that data survives a restart, and that the screens behave as designed. The scope covers all fourteen modules and the saved data file. It excludes the Tkinter library itself and the operating system.

9.2 Test strategy

The functional-programming architecture decides the strategy. Because the rules are pure functions, they can be tested exhaustively and cheaply with automated unit tests. The graphical shell has visual and timing behaviour that is better verified by a human, so it is covered by scripted manual tests.

Level	Target	Method	Automation	Status
Unit	fp_utils, fp_reducers, fp_analytics, fp_models	unittest, black-box and boundary values	Automated	6 tests exist; more designed in 9.8
Integration (data)	storage with models	Round trip and corrupted-file tests	Automated	1 test exists; more designed
Integration (controller)	main with reducers and storage	Drive callbacks directly, assert on state	Automatable with a hidden Tk root	Designed in 9.9
GUI functional	Each screen	Scripted manual test cases	Manual	Designed in Chapter 9 and Appendix A
System	Whole application on each operating system	End-to-end scenarios	Manual	Designed
Non-functional	Performance, resilience, usability, portability	Observation and measurement	Manual	Designed
Regression	All of the above	Re-run after every change	Automated part only	Command in 9.12

Techniques applied

Technique	Used for
Equivalence partitioning	Integrity bands, task priorities, anti-cheat modes, theme names
Boundary-value analysis	Integrity at 59, 60, 79, 80, 89, 90; due date equal to today; coins equal to cost
State-transition testing	Timer states (Chapter 7 figure)

Technique	Used for
Decision tables	Task completion bonus; unlock affordability
Error guessing	Corrupt JSON, empty title, invalid date, deleting an active task
Exploratory testing	Theme switching, window resizing, long sessions

Entry and exit criteria

Criterion	Definition
Entry	Code compiles and imports; Python 3 with Tk available
Exit (unit)	All unit tests pass; every public function of the core is covered by at least one test
Exit (GUI)	All high-priority manual cases pass on at least one platform; no open defect of severity Critical or High
Release	Exit criteria met and the known defects in Chapter 10 are either fixed or accepted in writing

9.3 Test environment

Item	Value
Automated run (for this report)	Python 3.12.3, standard unittest runner, Linux container, no display
Test command	python3 -m unittest -v test_app
Data under test	get_default_state() seed data; the supplied cozystudy_data.json for data observations
Display needed	No: test_app.py imports only the non-GUI modules
Recommended manual platforms	macOS (primary), Windows, Ubuntu with python3-tk

9.4 The existing automated suite

The file test_app.py defines one class, TestCozyStudyFP, with six tests. A setUp method gives each test a fresh default state.

Test 1: test_immutability

Aspect	Detail
Objective	Prove that records cannot be modified in place
Steps	Take the first task and try to assign a new title; take the first charm and try to assign equipped
Expected	Each assignment raises an exception
Result	Pass
Requirements	O12, NFR-03, NFR-04
Limits	Checks two classes of seven; the test accepts any Exception, which is broader than the precise FrozenInstanceError

Test 2: test_functional_combinators

Aspect	Detail
Objective	Verify pipe, curry, filter_tuple and map_tuple
Steps	pipe(5, double, add ten); curry of a three-argument sum called as f(1)(2)(3); filter evens from 1 to 6; double them
Expected	20; 6; (2, 4, 6); (4, 8, 12)
Result	Pass
Requirements	O12
Limits	compose, reduce_iterable, update_in_tuple, remove_from_tuple, clamp and format_seconds_to_min_sec are not tested directly

Test 3: test_pure_reducers

Aspect	Detail
Objective	Verify that reducers return new states and leave the original untouched
Steps	Complete task-2; check original coins are unchanged, new coins are higher, task count is unchanged and the task is marked completed. Add a new task; check the original has the old count and the new state has one more
Expected	As described
Result	Pass
Requirements	FR-22, FR-26, O12
Limits	Does not check the amount of the bonus, nor delete_task, unlock_charm, toggle_equip_charm, add_session, set_active_task or update_settings

Test 4: test_focus_integrity_calculation

Aspect	Detail
Objective	Verify the integrity formula
Steps	calculate_focus_integrity for (1500, 1500), (1125, 1500) and (0, 0)
Expected	100, 75, 100
Result	Pass
Requirements	FR-08
Limits	No rounding case other than exact values; no over-range ratio

Test 5: test_analytics_pipelines

Aspect	Detail
Objective	Smoke-test the analytics functions
Steps	Aggregate the seed sessions and check keys and positive focus minutes; build the daily timeline and check it is a list; build seven-day trends and check there are seven entries
Expected	As described
Result	Pass
Requirements	FR-28
Limits	Checks shapes, not values; the timeline for today is typically empty or small and is only checked for type

Test 6: test_storage_serialization

Aspect	Detail
Objective	Verify that state survives conversion to a dictionary and back
Steps	state_to_dict then dict_to_state; compare coins and the lengths of tasks and charms
Expected	Equal values
Result	Pass
Requirements	FR-30
Limits	Does not touch the file system, does not compare nested fields, and therefore cannot detect the last_active_date defect

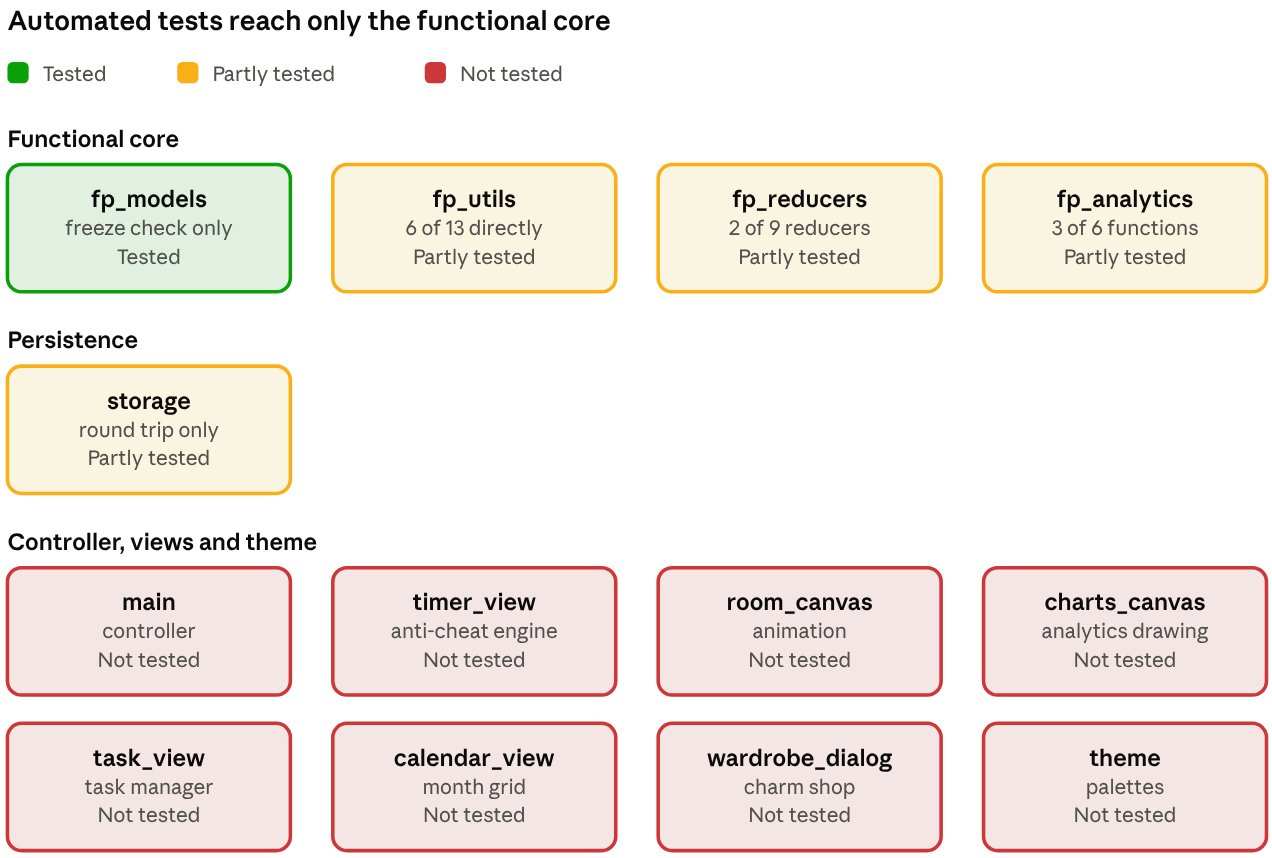
9.5 Execution results

The suite was run with the verbose runner.

Measure	Result
Tests run	6
Passed	6
Failed	0
Errors	0
Skipped	0
Elapsed time	0.003 seconds
Overall	OK

The speed is a direct benefit of the architecture: the whole business-logic suite finishes in milliseconds and needs no window.

9.6 Coverage assessment



test coverage map · 14 modules by status

The following table lists every function of the core and persistence layers and whether the existing tests exercise it. "Direct" means a test calls it; "indirect" means it runs only because another function calls it.

Module	Function	Existing coverage
fp_utils	pipe	Direct
fp_utils	compose	None
fp_utils	curry	Direct
fp_utils	map_tuple	Direct
fp_utils	filter_tuple	Direct (and indirect through analytics)
fp_utils	reduce_iterable	None

Module	Function	Existing coverage
fp_utils	find_first	Direct
fp_utils	update_in_tuple	Indirect (complete_task)
fp_utils	remove_from_tuple	None
fp_utils	clamp	Indirect (integrity)
fp_utils	calculate_focus_integrity	Direct
fp_utils	format_seconds_to_min_sec	None
fp_utils	calculate_charm_reward	None
fp_reducers	add_task	Direct
fp_reducers	update_task	None
fp_reducers	delete_task	None
fp_reducers	complete_task	Direct (no value check on the bonus)
fp_reducers	unlock_charm	None
fp_reducers	toggle_equip_charm	None
fp_reducers	add_session	None
fp_reducers	_compute_streak	None
fp_reducers	set_active_task	None
fp_reducers	update_settings	None
fp_analytics	get_today_sessions	None
fp_analytics	get_sessions_in_days	None
fp_analytics	compute_aggregate_integrity	Direct (shape)
fp_analytics	compute_daily_timeline	Direct (type)
fp_analytics	compute_7_day_trends	Direct (length)
fp_analytics	compute_task_statistics	None
storage	state_to_dict, dict_to_state	Direct (partial)
storage	load_state, save_state	None

Module	Function	Existing coverage
storage	get_default_* builders	Indirect (setUp)

Counting only direct calls, 13 of 38 functions are exercised, about a third by function count. The most important untested rules are the reward formula, the streak logic, unlocking charms and recording sessions, which are precisely the rules most tied to the application's central promise.

9.7 Gap between the test file's description and its content

The docstring at the top of test_app.py lists six areas. Comparing it with the code shows where the file promises more than it delivers.

Docstring claim	In the file?
Immutability and frozen dataclass integrity	Yes (test 1)
Pure FP combinators (pipe, curry, map_tuple, filter_tuple)	Yes (test 2)
Pure reducers (add_task, complete_task, unlock_charm, add_session)	Only add_task and complete_task
Higher-order analytics pipelines (integrity, daily timeline, 7-day trends)	Yes, shape checks only (test 5)
Storage load and save validation	Only dictionary round trip; no file I/O
GUI instantiation sanity check	No such test exists

The file also imports sys, os and datetime without using them. Correcting the docstring or adding the missing tests is recommended.

9.8 Designed unit test cases

The following cases extend the suite. Expected values marked (run) were confirmed by executing the code; the others are derived from the code and the stated rules. Failing cases are explicitly marked as defect checks.

9.8.1 fp_utils

ID	Function	Input	Expected
TC-U-01	format_seconds_to_min_sec	0	00:00
TC-U-02	format_seconds_to_min_sec	1500	25:00
TC-U-03	format_seconds_to_min_sec	59	00:59
TC-U-04	format_seconds_to_min_sec	-5	00:00 (negative values clamped)
TC-U-05	clamp	(0, 100, 150)	100
TC-U-06	clamp	(0, 100, -3)	0
TC-U-07	calculate_focus_integrity	(1420, 1500)	95 (run)
TC-U-08	calculate_focus_integrity	(1500, 1000)	100 (run; over-range clamped)
TC-U-09	calculate_charm_reward	(100, 1500, False)	20 (run)
TC-U-10	calculate_charm_reward	(90, 1500, False)	20 (run; boundary)
TC-U-11	calculate_charm_reward	(89, 1500, False)	15 (run; boundary)
TC-U-12	calculate_charm_reward	(80, 1500, False)	15 (run; boundary)
TC-U-13	calculate_charm_reward	(79, 1500, False)	8 (run; boundary)
TC-U-14	calculate_charm_reward	(60, 1500, False)	8 (run; boundary)

ID	Function	Input	Expected
TC-U-15	calculate_charm_reward	(59, 1500, False)	3 (run; boundary)
TC-U-16	calculate_charm_reward	(100, 1500, True)	22 (run)
TC-U-17	calculate_charm_reward	(59, 1500, True)	6 (run)
TC-U-18	calculate_charm_reward	(100, 50, False)	At least 2 (minimum reward)
TC-U-19	find_first	no match, default 'x'	'x'
TC-U-20	update_in_tuple	tuple (1, 2, 3), double evens	(1, 4, 3); original unchanged
TC-U-21	remove_from_tuple	tuple (1, 2, 3), remove odds	(2,)
TC-U-22	compose	compose(f, g)(x) with f = +1, g = x2, x = 5	11 (g first, then f)
TC-U-23	reduce_iterable	sum of (1, 2, 3) from 0	6

9.8.2 fp_reducers

ID	Reducer	Setup and action	Expected
TC-R-01	delete_task	Delete the task that is the active task	Task gone; active_task_id is None
TC-R-02	delete_task	Delete a task that is not active	Active id unchanged
TC-R-03	unlock_charm	Default state (85 coins), unlock charm_radio (75)	Success; coins 10 (run); charm unlocked and equipped
TC-R-04	unlock_charm	Default state, unlock charm_globe (160)	Failure; state unchanged (run)

ID	Reducer	Setup and action	Expected
TC-R-05	unlock_charm	Unlock an already unlocked charm	Failure; coins unchanged
TC-R-06	unlock_charm	Unknown charm id	Failure
TC-R-07	unlock_charm	Coins exactly equal to cost	Success; coins zero (boundary)
TC-R-08	toggle_equip_charm	Toggle an unlocked charm twice	Back to the original flag
TC-R-09	toggle_equip_charm	Toggle a locked charm	State unchanged
TC-R-10	add_session	Session linked to a todo task with 15 charms	Coins plus 15; task completed_cycles plus 1; status in_progress
TC-R-11	add_session	Session with task_id None	Tasks unchanged; coins increase
TC-R-12	add_session	Any session	New session is first in the tuple
TC-R-13	complete_task	On-time, low priority	Bonus 35
TC-R-14	complete_task	On-time, high priority	Bonus 45 (run for a high task due tomorrow)
TC-R-15	complete_task	Late, low priority	Bonus 20
TC-R-16	complete_task	Late, high priority	Bonus 30
TC-R-17	complete_task	Due date equal to today (boundary)	Counts as on time: plus 15
TC-R-18	complete_task	Unknown id	State unchanged, bonus 0

ID	Reducer	Setup and action	Expected
TC-R-19	complete_task	Task already completed (defect check D-04)	Specification says no second award; the code awards again (run: bonus 45 a second time, coins up by 90 in total)
TC-R-20	set_active_task	Set then clear	Value stored, then None
TC-R-21	update_settings	theme = midnight	New settings; other fields unchanged
TC-R-22	update_task	Change priority	Only that task changes

9.8.3 Streak logic (through add_session)

ID	Stored last active date	Current and best streak before	Expected after a session today
TC-S-01	Today	3 and 5	3 and 5 (unchanged)
TC-S-02	Yesterday	3 and 5	4 and 5
TC-S-03	Yesterday	5 and 5	6 and 6
TC-S-04	Two days ago	3 and 5	1 and 5 (reset)
TC-S-05	Empty or missing	1 and 1	1 and 1
TC-S-06	Yesterday, after a save and reload (defect check D-01)	3 and 5	4 and 5; the code gives 3 and 5 because the reloaded date is today (run: an old date of 2020-01-01 reloaded as today)

9.8.4 fp_analytics

ID	Function	Input	Expected
TC-A-01	compute_aggregate_integrity	Six seed sessions	8,500 active seconds, 800 distracted, 16 switches, 9,300 total, integrity 91 (run)
TC-A-02	compute_aggregate_integrity	Empty tuple	Zero totals, integrity 100
TC-A-03	compute_daily_timeline	Sessions with integrity 95, 80 and 60 on one date	Colours green, amber and red; sorted by start time
TC-A-04	compute_daily_timeline	Integrity 90 and 75 (boundaries)	Green and amber respectively
TC-A-05	compute_daily_timeline	No sessions on the date	Empty list
TC-A-06	compute_7_day_trends	Sessions only older than a week	Seven days, all zero focus minutes (run for the supplied data file)
TC-A-07	compute_7_day_trends	One session today of 1,420 active seconds	Last entry shows 23.7 focus minutes
TC-A-08	compute_task_statistics	The supplied data file	4 total, 1 completed, 3 overdue, 0 due today, 1 high-priority open, 25 percent (run)
TC-A-09	Task.is_overdue	Due yesterday, status todo	True
TC-A-10	Task.is_overdue	Due yesterday, status completed	False
TC-A-11	Task.is_due_today	Due today, status todo	True

9.8.5 storage

ID	Scenario	Expected
TC-D-01	File missing	Default state returned and file created
TC-D-02	File contains text that is not JSON	Error printed; default state returned and saved
TC-D-03	File is valid JSON but empty object	Defaults for subjects and charms; no tasks or sessions
TC-D-04	Settings contains an unknown key	Key ignored; no crash
TC-D-05	Task object missing a required field	Exception caught; defaults returned
TC-D-06	Save then load	Equal coins, tasks, charms, sessions and settings
TC-D-07	Save then load, last_active_date compared (defect check D-01)	Equal; the code returns today's date instead
TC-D-08	Emoji in titles	Preserved after a round trip
TC-D-09	Read-only folder	save_state returns False; no crash
TC-D-10	New charm added to the code, old data file loaded	The stored charm list replaces the defaults, so the new charm does not appear (migration gap, see 11.6)

9.9 Sample test code for the missing tests

The snippets below can be added to test_app.py. The first two pass today. The third demonstrates defect D-01 and will fail until the defect is fixed.

```
def test_unlock_charm_deducts_cost(self):
    new_state, ok = fp_reducers.unlock_charm(self.state, 'charm_radio')
    self.assertTrue(ok)
    self.assertEqual(new_state.coins, self.state.coins - 75)
    radio = fp_utils.find_first(lambda c: c.id == 'charm_radio',
new_state.charms)
    self.assertTrue(radio.unlocked and radio.equipped)

def test_reward_bands(self):
    reward = lambda i, strict: fp_utils.calculate_charm_reward(i, 1500,
strict)
    self.assertEqual([reward(i, False) for i in (100, 89, 79, 59)], [20, 15,
8, 3])
    self.assertEqual([reward(i, True) for i in (100, 89, 79, 59)], [22, 18,
10, 6])

def test_last_active_date_survives_reload(self):
    from dataclasses import replace
    old = replace(self.state, stats=replace(self.state.stats,
last_active_date='2020-01-01'))
    restored = storage.dict_to_state(storage.state_to_dict(old))
    self.assertEqual(restored.stats.last_active_date, '2020-01-01')  # fails
today
```

A GUI-free controller test is possible by creating the root window with withdraw(), instantiating CozyStudyApp with a temporary STORAGE_FILE, and calling its callback methods directly, asserting on app.state. This needs a display or a virtual display such as Xvfb on a build server.

9.10 Designed GUI test cases (manual)

GUI behaviour depends on windows, focus and timing, so these cases are written as manual scripts. Each has a unique identifier, the steps and the expected result. They have been designed from the code and have not yet been executed on a machine with a display. Cases that probe known defects say so.

9.10.1 Start-up and navigation

ID	Steps	Expected result	Requirement
TC-G-01	Delete cozystudy_data.json and launch	Window opens on the Room tab; data file is created; balance shows 85; streak badge shows 3 days	FR-30
TC-G-02	Launch with an existing data file	Saved balance, theme and tasks appear	FR-30
TC-G-03	Click each of the six tab buttons in turn	The matching screen is shown; the active button is highlighted, others muted	FR-33
TC-G-04	Press keys 1 to 5, then 6	Room, Timer, Analytics, Tasks, Calendar appear in turn; key 6 does nothing	FR-33
TC-G-05	Drag the window to its smallest size	Cannot go below 850 by 580; every screen is still usable	NFR-07
TC-G-06	Enlarge the window to full screen	Room scene scales; other screens remain correct, with extra margin	NFR-07
TC-G-07	Change something (for example complete a task), close, relaunch	The change persists	FR-30
TC-G-08	Launch from a different current directory (for example python3 /path/main.py from the home folder)	Starts normally and uses the data file next to the program	NFR-01

9.10.2 Focus timer and anti-cheat

ID	Steps	Expected result	Requirement
TC-G-09	Open the Timer tab	Ring full, digits 25:00, integrity 100 percent, 0 switches	FR-01, FR-03

ID	Steps	Expected result	Requirement
TC-G-10	Press Start	Button changes to Pause; digits count down one second at a time; green arc grows clockwise from the top	FR-04
TC-G-11	Press Pause, wait, press Resume	Countdown holds then continues; captions change accordingly	FR-04
TC-G-12	Run 30 seconds, press Reset	Digits return to 25:00; counters are zero; no session is added anywhere	FR-04
TC-G-13	Press Short Break	05:00; ring in accent colour; button reads Start Break; integrity not affected by break time	FR-02
TC-G-14	Press Long Break	15:00	FR-02
TC-G-15	Honest mode: Start, switch to another application for 10 seconds, return	Timer never stops; tab switches 1; about 10 seconds off-focus; pill red then green; ring red while away	FR-05, FR-06, FR-07
TC-G-16	Honest mode with sound on, then off: lose focus	Alert sound when on; silence when off (bell on non-macOS systems)	FR-10
TC-G-17	Strict mode: Start, then switch away	Timer pauses; warning dialog; button reads Resume (Paused by Anti-Cheat); switches 1	FR-09
TC-G-18	Strict mode: return and press Resume	Countdown continues from the paused value	FR-09

ID	Steps	Expected result	Requirement
TC-G-19	Complete a full 25-minute honest session with no distraction (or temporarily set pomodoro_minutes to a small value in the data file for testing)	Summary box shows 25 minutes focus, 0 off-tab, integrity 100 percent, reward 20; balance rises by 20	FR-11, FR-12
TC-G-20	Complete a session with about one quarter of the time away (integrity about 75)	Reward 8 in honest mode	FR-11
TC-G-21	Complete a perfect session in strict mode	Reward 22	FR-11
TC-G-22	Select a task before starting, then complete	That task's cycle counter increases by 1; a todo task becomes in progress	FR-13
TC-G-23	Choose General Study, then complete	No task changes; session titled General Study	FR-12
TC-G-24	Change the anti-cheat mode in Settings, then look at the Timer tab (defect check D-02)	Intended: the same mode appears; actual: the Timer's own selector keeps its previous value	FR-09
TC-G-25	On the Tasks tab press Focus Now on a task that comes after a completed task in the list (defect check D-03)	Intended: Timer opens with that task selected; actual: an index error may occur or the wrong task is selected	FR-25
TC-G-26	Start a session and close the application	No session is recorded; the next launch shows no trace (known design limit D9)	FR-12

9.10.3 Room

ID	Steps	Expected result	Requirement
TC-G-27	Open Room with seed data	Matcha cup with steam, lamp with light, sleeping cat, succulent, fairy lights, bookshelf, desk and laptop all visible	FR-15, FR-16
TC-G-28	Click on the cat	A speech bubble with one of four messages appears and disappears after about four seconds	FR-18
TC-G-29	Unequip the cat in the Wardrobe, then click where it was	Nothing happens	FR-18
TC-G-30	Click the lamp shade twice	Light cone disappears then returns	FR-18
TC-G-31	Resize the window repeatedly	Scene rescales without flicker or errors	NFR-07
TC-G-32	Switch theme to Midnight, then to Sakura	Midnight shows moon and twinkling stars; light themes show sun and drifting cloud	FR-17, FR-32
TC-G-33	Watch for 30 seconds	Steam rises, cat breathes, fairy lights flicker, cloud drifts	FR-17
TC-G-34	Read the header	Streak badge shows the streak; button shows the balance	FR-14

9.10.4 Wardrobe

ID	Steps	Expected result	Requirement
TC-G-35	Open the Wardrobe from the Room button, the balance pill and the Wardrobe button	The same dialog appears each time with four category tabs	FR-19

ID	Steps	Expected result	Requirement
TC-G-36	Compare the dialog balance with the navbar	Equal	FR-19
TC-G-37	With 85 charms, unlock the radio	Confirmation box; balance 10; radio appears on the desk; dialog closes	FR-20
TC-G-38	Press Unlock on the Floating Moon Globe (160)	Information box says 160 charms are needed; nothing changes	FR-20
TC-G-39	Unequip an owned charm	It disappears from the room; button now reads Equip	FR-21
TC-G-40	Equip it again	It reappears; the In Room tag shows	FR-21
TC-G-41	While the dialog is open, click the main window	The main window does not respond (modal)	NFR-07
TC-G-42	Open Desk Decor and scroll	The six desk charms are listed; the scroll bar works	FR-19

9.10.5 Tasks

ID	Steps	Expected result	Requirement
TC-G-43	Open Tasks with seed data	Four active tasks	FR-23
TC-G-44	Filter High	Two tasks	FR-23
TC-G-45	Filter Medium	Two tasks	FR-23
TC-G-46	Filter Low	Friendly all-caught-up message (the only low task is completed)	FR-23
TC-G-47	Filter Completed	One task with a struck-through title and no action buttons except delete	FR-23

ID	Steps	Expected result	Requirement
TC-G-48	Add a task with a valid title	It appears first in the list and survives a restart	FR-22
TC-G-49	Press Save with an empty title	Error box; the dialog stays open	FR-22
TC-G-50	Enter a due date of 2026-13-45 (defect check D-09)	Intended: rejected; actual: accepted and shown as is	FR-22
TC-G-51	Create tasks due yesterday, today and next week	Labels: Overdue with date, Due Today, and Due with date	FR-24
TC-G-52	Complete a high-priority task due tomorrow	Message: +45; balance rises by 45; task leaves the active list	FR-26
TC-G-53	Complete a low-priority overdue task	Message: +20	FR-26
TC-G-54	Press the delete cross, answer No, then Yes	Task remains, then disappears	FR-27
TC-G-55	Delete the task that was chosen with Focus Now	Timer's task selector returns to General Study	FR-27
TC-G-56	Look at the subject badge of each card	Icon and subject name; General if the subject is unknown	FR-22

9.10.6 Analytics

ID	Steps	Expected result	Requirement
TC-G-57	Open Analytics with seed data	Pure focus 2.4 hours; integrity 91 percent with 16 switches; 6 cycles; streak 3 with best 5	FR-28
TC-G-58	Read the donut	Centre 91 percent; legend Focus 141.7 m, Off-tab 13.3 m, switches 16	FR-28
TC-G-59	Read the seven-day chart	Stacked bars on the days that have sessions; zero days show a thin grey stub; minute labels on top	FR-28

ID	Steps	Expected result	Requirement
TC-G-60	Read the timeline	Today's sessions in time order; 95 percent green, 87 percent amber	FR-28
TC-G-61	Use a data file without sessions	Donut shows a grey ring and No Data; the timeline shows the empty message	FR-28
TC-G-62	Complete a session, then open Analytics	Figures and bars reflect it immediately	FR-28

9.10.7 Calendar

ID	Steps	Expected result	Requirement
TC-G-63	Open Calendar	Current month; today highlighted and selected	FR-29
TC-G-64	Press next repeatedly from December, then previous	Month and year roll over correctly in both directions	FR-29
TC-G-65	Look at the grid	Green dot on days with sessions; red dot on days with unfinished deadlines	FR-29
TC-G-66	Click a day with sessions	Inspector shows total focus, session count, switches and each session	FR-29
TC-G-67	Click a day without data	Inspector says no sessions were recorded	FR-29
TC-G-68	Click a deadline day	Deadlines section lists the task titles	FR-29
TC-G-69	Select a day, change month, change back	The selected highlight returns	FR-29
TC-G-70	Complete a session, then open Calendar	Today shows the new dot and session	FR-29

9.10.8 Settings and data management

ID	Steps	Expected result	Requirement
TC-G-71	Choose each theme in turn	Window, navbar, tab buttons and rebuilt lists change; screens built earlier may keep some old colours until restart (known D-07)	FR-32
TC-G-72	Choose a theme, restart	The whole application opens in that theme	FR-32
TC-G-73	Change anti-cheat mode, restart	The saved choice appears in Settings (see D-02 for the timer)	FR-09
TC-G-74	Turn sounds off	No alert or chime; setting persists	FR-10
TC-G-75	Export Backup, save the file	A JSON file with the same content as the data file	FR-31
TC-G-76	Export Backup, cancel the dialog	Nothing is written	FR-31
TC-G-77	Reset, answer No, then Yes	No change, then seed data with 85 charms	FR-31
TC-G-78	After a reset, restart	Sakura theme and all seed data	FR-31

9.11 System-level scenario with expected numbers

The scenario below links several features and gives expected numbers calculated with the verified formulas. Start from a fresh seed state (no data file).

Step	Action	Expected balance	Notes
1	Launch	85	Seed balance
2	Run a 25-minute honest session at 96 percent integrity	105	Reward 20 (integrity 90 or more)

Step	Action	Expected balance	Notes
3	Complete Linear Algebra Problem Set 4 (high, due tomorrow)	150	Bonus 20 plus 15 on time plus 10 high priority equals 45
4	Unlock the Vintage Lo-Fi Radio	75	Cost 75; radio appears on the desk
5	Run a 25-minute strict session at 100 percent	97	Reward 22
6	Try to unlock the Floating Moon Globe	97	Needs 160; refused
7	Open Analytics		Seven sessions, aggregate integrity recalculated from stored values
8	Open Calendar and select today		Today's sessions and a streak that has not decreased
9	Export a backup, reset, then restart	85	Back to seed

Step 3's task also has an effect on the timer's target list: it disappears because completed tasks are filtered out.

9.12 Non-functional test cases

ID	Area	Procedure	Acceptance criterion
TC-N-01	Start-up time	Time from launch to a drawn Room screen on a typical laptop	Under 2 seconds
TC-N-02	CPU use	Leave the Room tab open for 5 minutes and observe CPU	Low single-digit percent of one core; note any growth
TC-N-03	CPU use, hidden	Switch to another tab for 5 minutes and observe CPU	Ideally lower than TC-N-02; if equal the animation loop is running while hidden (see D-15)

ID	Area	Procedure	Acceptance criterion
TC-N-04	Memory	Leave the application running for an hour with the timer running	Memory does not grow steadily
TC-N-05	Large data	Create a data file with 5,000 sessions and 500 tasks	Every screen opens in under 2 seconds
TC-N-06	Resilience	Replace the data file with random text	Application starts with seed data; message on the console
TC-N-07	Resilience	Make the data folder read-only, then complete a task	No crash; data not saved (the user is not told, see D-08)
TC-N-08	Portability	Run on macOS, Windows and Ubuntu	All functions work; sound falls back to the bell off macOS
TC-N-09	Clock change	Set the system date forward one day and complete a session	Streak logic behaves as in TC-S-02
TC-N-10	Sleep	Start the timer, sleep the computer for 10 minutes, wake	Document the result; expected to show only a few seconds elapsed because ticks, not wall time, are counted (see D-06)
TC-N-11	Accessibility	Check text legibility and colour-only cues in each theme	All status information is also conveyed by words or digits
TC-N-12	Security	Edit the data file to put script-like text in a title	Text appears literally; nothing is executed

9.13 Usability test plan

Five participants who are students perform the following tasks without help: start a 25-minute session linked to an existing task; find out how many minutes they were distracted; buy a charm; add a task due next week; find what they studied last Tuesday. Success rates, time on task and comments are recorded. The test would show whether the label "Focus Integrity" is understood, whether the Calendar is discovered as the place to review past days, and whether the two anti-cheat modes are distinguished.

9.14 Test summary

Level	Cases	Executed	Passed	Notes
Automated unit and data (existing)	6	6	6	Run for this report
Designed unit and data cases	72	19 marked (run)	17	Of the 19 marked cases, 17 matched the expected value and 2 reproduced defects D-01 and D-04; the rest are not yet automated
Designed GUI manual cases	78	0		Require a machine with a display
System scenario	1 (9 steps)	0		
Non-functional cases	12	0		

The honest summary is that the functional core is sound and its few tests pass, that the main numeric rules were additionally checked by running the code, and that the interactive layer remains to be verified by the designed manual cases.

9.15 Defects found during testing and analysis

While running the code and designing cases, the following defects were confirmed or strongly suspected. They are described in full in Chapter 10.

Id	Summary	How established
D-01	Streak date reset on every load	Run: an old date reloads as today; the supplied data file shows the symptom
D-04	Completing an already completed task awards the bonus again	Run: second bonus of 45
D-02	Settings anti-cheat choice does not reach the timer	Code inspection
D-03	Focus Now selects the wrong index when completed tasks exist	Code inspection

Id	Summary	How established
D-05 to D-16	Lower-severity issues	Code inspection

9.16 Regression and continuous testing

Recommended command for every change:

```
python3 -m unittest -v test_app
```

Because the suite needs no display it runs on any build server. A minimal workflow file for a hosted build service installs Python, checks out the repository and runs the command above. GUI cases should be re-run manually before each release, concentrating on those linked to changed modules using the traceability matrix in Appendix B.

9.17 Recommendations from the test analysis

1. Add the missing unit tests for the reward formula, unlocking, session recording and streak logic (cases TC-U-09 to TC-U-18, TC-R-03 to TC-R-12, TC-S-01 to TC-S-06).
2. Add a file-based storage test using a temporary directory (TC-D-01 to TC-D-09).
3. Fix D-01 and D-04 and add their regression tests, which fail today.
4. Correct the test file's docstring or implement the promised GUI sanity test.
5. Execute the manual cases on one platform and record results in the sheet in Appendix A.

10. Defects, Risks and Code-Review Findings

This chapter records what a careful review of the delivered code and data revealed. It is written to help the project owner decide what to fix, not to criticise a well-structured piece of work. The architecture is sound and most findings are small. Each finding has an identifier, a severity, the evidence, the impact and a proposed fix. Evidence is labelled **run** when it was reproduced by executing the code, and **inspection** when it follows from reading the code and has not been executed in a window.

10.0 Summary of findings

Id	Title	Severity	Evidence	Module
D-01	Streak date is reset to today on every load	High	Run	storage
D-02	Anti-cheat mode in Settings does not reach the timer	High	Inspection	main, timer_view
D-03	Focus Now selects the wrong item or fails after a completed task	High	Inspection	timer_view
D-04	Completing a completed task awards the bonus again	Medium	Run	fp_reducers
D-05	Selected task found by title substring	Low	Inspection	timer_view
D-06	Timer counts ticks, not elapsed time	Medium	Inspection	timer_view
D-07	Theme change restyles views only partly	Medium	Inspection	main and views
D-08	Save failures are silent and writes are not atomic	Medium	Inspection	storage
D-09	Due date and cycle inputs are not validated	Medium	Inspection	task_view
D-10	Colour thresholds for integrity differ between screens	Low	Inspection	timer_view, fp_analytics
D-11	README statements that differ from the code	Low	Inspection	README
D-12	Unused code and declarations	Low	Inspection	several
D-13	Break lengths cannot be changed; captions are fixed	Low	Inspection	timer_view
D-14	Wardrobe closes after every action	Low	Inspection	wardrobe_dialog
D-15	Room animation keeps running when hidden	Low	Inspection	room_canvas

Id	Title	Severity	Evidence	Module
D-16	The app's own dialogs may count as tab switches	Medium	Inspection	main, timer_view
D-17	New charms never reach existing users	Medium	Inspection	storage
D-18	Number-key shortcuts could clash with future text fields	Low	Inspection	main
D-19	Task and session ids can collide within one second	Low	Inspection	task_view, timer_view
D-20	An unfinished session is lost when the window closes	Low	Inspection	timer_view

10.1 D-01: Streak date is reset to today on every load (High)

Where. storage.dict_to_state.

What happens. When the stats record is rebuilt, the code keeps only keys for which `hasattr(UserStats, key)` is true. The field last_active_date is declared with `field(default_factory=...)`. For such fields the dataclass machinery removes the class attribute, so hasattr returns False and the stored date is thrown away. The new UserStats then receives a fresh default, which is today's date.

Evidence (run).

Check	Result
hasattr(UserStats, 'last_active_date')	False
hasattr(Settings, 'theme')	True (plain default values stay as class attributes)
Data file with last_active_date set to 2020-01-01, then loaded	Loaded value is today's date
Supplied cozystudy_data.json	Stores 2026-10-16 even though the newest session is from 13 September 2026

Impact. The streak logic compares the stored date with today. After every restart the stored date equals today, so _compute_streak returns the stats unchanged and the streak never advances. The streak badge, the analytics card and best streak are frozen at their stored values (3 and 5 in the supplied file). The core feature of a daily streak therefore does not work across sessions, although the reducer itself is correct.

Proposed fix. Filter by declared fields instead of class attributes:

```
from dataclasses import fields

def _known_fields(cls, raw):
    names = {f.name for f in fields(cls)}
    return {k: v for k, v in raw.items() if k in names}

settings = Settings(**_known_fields(Settings, data.get('settings', {})))
stats = UserStats(**_known_fields(UserStats, data.get('stats', {})))
```

Regression test. TC-D-07 and TC-S-06.

10.2 D-02: Anti-cheat mode in Settings does not reach the timer (High)

Where. main._on_cheat_mode_change, timer_view.

What happens. The timer view creates its own StringVar from the stored mode at start-up and reads that variable when a window loses focus or a session ends. Choosing a different mode in Settings updates the application state but never updates the timer's variable, and update_state in the timer view does not copy the mode. Conversely, the handler of the timer's own selector (`_on_anti_cheat_change`) contains only `pass` , so a choice made on the timer screen is not saved or reported.

Impact. The two selectors can disagree. A student who selects Strict in Settings may still be running in Honest mode and receive honest-mode behaviour and rewards without being told. The stored setting does not describe what actually ran.

Proposed fix. Make the application state the only source of truth. Either remove the duplicate selector from the timer screen, or give the timer view a callback to the controller and copy the value in update_state:

```
def update_state(self, state):
    self.state = state
    self.anti_cheat_var.set(state.settings.anti_cheat_mode)
    ...

def _on_anti_cheat_change(self, event=None):
    self.on_mode_change(self.anti_cheat_var.get())   # controller calls
update_settings
```

10.3 D-03: Focus Now selects the wrong item or fails after a completed task (High)

Where. timer_view.set_active_task and _refresh_task_selector_options.

What happens. The drop-down lists General Study followed by only the unfinished tasks. set_active_task, however, loops over all tasks, including completed ones, and selects position index plus one. After a completed task in the list the position is too large. With the default seed data the completed task is fourth, so Focus Now on the fifth task asks for position 5 in a list that has positions 0 to 4. With the supplied data file the completed task is third and the fourth task asks for position 4 in a list with positions 0 to 3. The widget raises a TclError for an index out of range.

Impact. The error occurs inside the controller's callback before the tab is switched, so pressing Focus Now appears to do nothing, and a traceback is printed to the console. For tasks that come before the completed one the selection is correct, which makes the bug intermittent and harder to notice.

Proposed fix. Number only the unfinished tasks:

```
def set_active_task(self, task_id):
    self._refresh_task_selector_options()
    active = [t for t in self.state.tasks if t.status != 'completed']
    for idx, task in enumerate(active):
        if task.id == task_id:
            self.task_selector.current(idx + 1)
            break
```

10.4 D-04: Completing a completed task awards the bonus again (Medium)

Where. fp_reducers.complete_task.

Evidence (run). Completing task-2 (high priority, due tomorrow) gave a bonus of 45. Completing it a second time gave another 45, raising the balance by 90 in total.

Impact. The graphical interface hides the Done button for completed tasks, so a user cannot trigger this today. Any future feature that calls the reducer (an undo, a keyboard shortcut, an import) could create free currency. A pure reducer should defend its own invariants.

Proposed fix. `if not task or task.status == 'completed': return state, 0`.

10.5 D-05: Selected task found by title substring (Low)

The timer decides which task a session belongs to by testing whether each unfinished task's title appears inside the drop-down text. If one title is a substring of another (for example Essay and Essay draft 2), the first match wins and the session may be attributed to the wrong task. A mapping from drop-down index to task id would remove the ambiguity.

10.6 D-06: Timer counts ticks, not elapsed time (Medium)

The loop subtracts one second each time after(1000) fires. If the computer sleeps, the process is suspended, or a modal dialog delays the event loop, the countdown falls behind real time. Storing the start time and computing remaining time from the clock on every tick would make the displayed time correct and would let the engine count the real number of seconds the window was unfocused. The same change would make long sessions immune to small timer drift.

10.7 D-07: Theme change restyles views only partly (Medium)

Each view builds its widgets once, with colours read from the theme at construction time. When the theme changes, update_state replaces the stored theme dictionary and the page background, and views that rebuild their content (tasks, calendar inspector, analytics cards, wardrobe dialog when reopened) pick up the new colours. Headers, buttons, labels and canvases created in the constructors keep their old colours until the program is restarted. The settings screen is built once at start-up. A student switching from Sakura to Midnight therefore sees a mixture of palettes. Possible fixes are to rebuild all views on a theme change, or to tell the user that a restart is needed.

A related minor point: Reset to Starter Demo Data restores the Sakura theme in the state while the combo box on the Settings screen still shows the previous theme.

10.8 D-08: Save failures are silent and writes are not atomic (Medium)

save_state catches every exception, prints it to the console and returns False; no caller reads the result. If the disk is full or the folder is read-only the user continues to earn rewards that will vanish after a restart. The file is also opened for writing directly, so a crash in the middle of a write can leave a truncated, invalid file, which the loader would then replace with seed data and lose the student's history.

Proposed fix.

```
def save_state(state):
    tmp = STORAGE_FILE + '.tmp'
    try:
        with open(tmp, 'w', encoding='utf-8') as f:
            json.dump(state_to_dict(state), f, indent=2)
        os.replace(tmp, STORAGE_FILE)      # atomic on all platforms
        return True
    except OSError as e:
        print(f'Failed to save: {e}')
        return False
```

The controller should show a warning message when the function returns False. Keeping one rolling backup copy (for example cozystudy_data.json.bak) would also protect against bad loads.

10.9 D-09: Due date and cycle inputs are not validated (Medium)

The due-date box is free text. An invalid date such as 2026-13-45 or tomorrow is stored and compared as text with today's date, which gives wrong overdue labels and wrong bonuses. The estimated-cycles spin box accepts typed non-numeric text, and int() on it would raise an exception in the save handler. Validation with datetime.strptime and a numeric check, with an error box like the one for an empty title, would close the gap.

10.10 D-10: Colour thresholds for integrity differ between screens (Low)

Place	Green	Amber	Red
Live meter on the timer	85 and above	70 to 84	Below 70
Analytics timeline	90 and above	75 to 89	Below 75
Reward multiplier bands	90 and above	80 to 89 (and 60 to 79 at half)	Below 60

Three sets of numbers describe one idea. Defining the bands once in fp_utils and using them everywhere would make the screens consistent and the README easier to keep accurate.

10.11 D-11: README statements that differ from the code (Low)

README statement	What the code does
Star Charms are earned when integrity is at least 80 percent	Charms are earned at every level; 79 to 60 percent earns about half the base and below 60 about a fifth
The window shows day or night	The sky depends on the chosen theme, not on the clock
Pipelines using pipe, compose, curry and reduce_iterable	Only filter_tuple, update_in_tuple, find_first and functools.reduce are used in production code
Amber timeline band is 75 to 89 percent	Correct for the timeline, but the live meter uses different bands
Task completion bonus is shown as +20 on the button	The actual bonus is 20 to 45
Test suite verifies reducers including unlocking	Only add_task and complete_task are tested

The label on the Done button always says plus 20 charms, but the bonus is usually 35 to 45 for an on-time task, so the screen understates the reward.

10.12 to 10.20: Lower-severity findings

Id	Detail	Suggested remedy
D-12	Unused functions and declarations listed in Section 8.12	Use, test or remove them
D-13	The captions 5 minutes and 15 minutes on the break buttons are fixed text while the durations come from settings, and Settings has no control for them	Build captions from the settings values and add two spin boxes
D-14	The wardrobe closes after each unlock or equip, so decorating a room needs repeated opening	Refresh the dialog in place instead of destroying it
D-15	The room redraws 10 times a second even when another tab is visible, because the loop is never stopped (to be confirmed by TC-N-03)	Skip drawing when the view is not mapped, or cancel the loop on tab change
D-16	Tk reports a focus loss on the main window when its own dialogs (Wardrobe, Add Task, message boxes) take focus; if a session is running this would count as a tab switch and, in strict mode, pause it	Ignore focus changes while an application dialog is open; to be confirmed manually
D-17	dict_to_state uses the stored list of charms (or subjects) whenever it is not empty, so a charm added to the code later never appears for an existing user	Merge stored and default catalogues by id when loading, and add a data version number
D-18	Keys 1 to 5 are bound on the main window; harmless today because the main window has no text field, but a future entry box would switch tabs while typing	Bind only when the focused widget is not an entry
D-19	Ids are built from the Unix time in whole seconds	Use uuid4 hex strings
D-20	Closing the window mid-session discards the elapsed time	Offer to save a partial session on close

10.21 Prioritised fix plan

Priority	Items	Reason	Effort
1	D-01, D-04	Small, safe fixes; D-01 restores a headline feature; both have failing regression tests ready	About an hour
2	D-03, D-02	Visible bugs in the core flow from tasks to timer	Half a day
3	D-08, D-09, D-16	Protect data and honesty of measurements	Half a day
4	D-06, D-07, D-17	Larger refactors with user-visible benefits	One to two days
5	D-05, D-10 to D-15, D-18 to D-20	Polish and consistency	Spread over later releases

10.22 Project risk register

Risk	Likelihood	Impact	Mitigation
Students defeat the anti-cheat by staying in the app window while doing something else	High	Medium	Present integrity as a self-honesty tool; optional strict mode; do not rank users
Data loss from a failed or interrupted write	Low	High	Atomic writes and a backup copy (D-08)
Differences between operating systems (fonts, emoji, sound)	Medium	Low	Test on all three; keep fallbacks
Frozen streak undermines motivation	High (today)	Medium	Fix D-01
Technical debt from unused code	Medium	Low	Tidy during the next release
Single-developer knowledge	Medium	Medium	This report and the module docstrings

Risk	Likelihood	Impact	Mitigation
Growth of data file	Low	Low	Archive old sessions after a year

10.23 What the review found to be strong

A fair report also records what works well.

Strength	Evidence
Clear separation of concerns	The core never imports tkinter; a dependency table with no cycles
Immutability is real, not nominal	Frozen dataclasses and a test that proves it
Rules are small and readable	Each reducer is a few lines with a docstring
Defensive loading	A corrupt file cannot prevent start-up
Thoughtful anti-cheat design	The honest mode measures instead of punishing; the strict mode is optional
Cohesive visual design	Shared tokens, four palettes, animated room
Zero dependencies	Easy to run and to hand in
Useful seed data	Every screen has content on first launch

11. Phase 7: Deployment, Operation and Maintenance

The last phase covers getting the software onto a machine, running it day to day, keeping data safe, and changing it over time.

11.1 Installation

CozyStudy is distributed as a folder of source files. There is nothing to compile.

Platform	Prerequisites	Steps
macOS	Python 3 from python.org or Homebrew (Tk is included)	Copy the folder; open Terminal in it; run python3 main.py or sh run.sh
Windows	Python 3 from python.org with the tcl/tk option ticked	Copy the folder; open a command prompt in it; run python main.py
Ubuntu and Debian	sudo apt install python3 python3-tk	Copy the folder; run python3 main.py
Fedora	sudo dnf install python3 python3-tkinter	Copy the folder; run python3 main.py

To confirm that Tk works, run python3 -m tkinter; a small test window should open. Emoji display best with a system emoji font; on older Linux systems install one such as Noto Color Emoji.

11.2 The launcher script (run.sh)

Line of logic	Purpose
DIR=... cd "$DIR"	Resolve the folder containing the script and change into it, so the program works from any starting directory
if command -v python3	Use the python3 found on the path
elif the python.org 3.14 framework path exists	Fall back to that interpreter on macOS
else echo message	Tell the user that Python 3 is required

The fallback path is specific to one Python version. A more general script would try python3, then python, then a list of known locations.

11.3 First run and normal operation

On the first run no data file exists, so the program writes seed data (85 charms, five tasks, six sessions, a streak of 3). From then on, every change is saved immediately. There is no save button and no shutdown step. The program uses no network and no background

services, and it stops completely when the window closes.

11.4 Backup and restore

Task	How
Back up	Settings, Export Backup (JSON), choose a location; or copy cozystudy_data.json while the program is closed
Restore	Close the program, replace cozystudy_data.json with the backup, start the program. There is no import button, so this is done by file copy
Move to another computer	Copy the folder, or just the data file into a fresh copy of the program
Start over	Settings, Reset to Starter Demo Data (asks for confirmation); or delete the data file

Because the backup is the same format as the live file, a backup can be inspected or edited in any text editor. Editing by hand is possible but a typing error can make the file invalid, in which case the program starts with seed data and overwrites the broken file; keep a copy first.

11.5 Troubleshooting guide

Symptom	Likely cause	Remedy
No module named tkinter	Tk not installed	Install python3-tk (Linux) or reinstall Python with Tk
Window opens but emoji show as squares	No emoji font	Install an emoji font
Application starts with demo data after you had your own	Data file was unreadable or in another folder	Look for the console message; restore a backup; make sure you start the program from the same folder
Streak does not grow	Known defect D-01	Apply the fix in 10.1

Symptom	Likely cause	Remedy
Focus Now does nothing	Known defect D-03	Apply the fix in 10.3 or select the task in the drop-down by hand
No sound	Sound disabled, or not macOS	Enable in Settings; on other systems only the Tk bell plays
Theme looks mixed after switching	Known behaviour D-07	Restart the application
Timer shows less time passed after sleep	Known behaviour D-06	Avoid sleeping during a session
Changes lost after restart	Save failed (read-only folder or full disk)	Check the console output and folder permissions

11.6 Data versions and migration

The data file has no version number. Loading replaces the default list of charms or subjects by the stored list whenever the stored list is not empty. Therefore a charm added to the program after a user's first run does not appear for that user (defect D-17), and renamed or removed fields could break older files. A safe migration approach is to add a "version" key at the top of the file, and on load to merge stored and built-in charms by id (keeping the stored unlocked and equipped flags), apply conversion steps for old versions, and then save with the new version.

11.7 Maintenance activities

Type	Meaning	Examples for CozyStudy
Corrective	Fix faults	D-01 to D-04
Adaptive	Keep working as platforms change	New Python or Tk versions; new operating system sound locations; Windows and Linux sound support
Perfective	Improve and extend	Grace period, editable breaks, task statistics panel
Preventive	Reduce future risk	More unit tests, atomic saves, data versioning, remove dead code

11.8 Extension guide

Add a new charm

1. Add a Charm entry to get_default_charms in storage.py with a unique id, category, icon, cost and description.
2. Add a drawing branch in room_canvas._draw_equipped_charms that tests is_charm_equipped with the new id.
3. Choose a position as a fraction of the desk, wall or floor geometry (see Chapter 7).
4. Existing users will not see it until the migration in 11.6 is added.
5. Add a test that the new id is present and costs what you expect.

Add a new theme

1. Add an entry to THEMES in theme.py with all twenty-two keys.
2. Add its key to the list of values for the combo box in the Settings screen.
3. If the theme is dark, extend the places in room_canvas that test for midnight (sky colour, desk colour, frame colour, laptop screen).

Add a new reducer

1. Write a pure function in fp_reducers.py that takes the state and returns a new state, using replace and the tuple helpers.
2. Write unit tests first.

3. Add a callback in main.py that calls it, saves and synchronises.
4. Pass the callback to the view that needs it.

Add a new analytics chart

1. Write a pure function in fp_analytics.py that returns plain dictionaries.
2. Add a card and canvas in charts_canvas.py and a render method called from redraw_all.

Add a new screen

1. Create a frame class with an update_state method.
2. Add it to the views dictionary and tab definitions in main.py.
3. Add a shortcut key if desired.

11.9 Release checklist

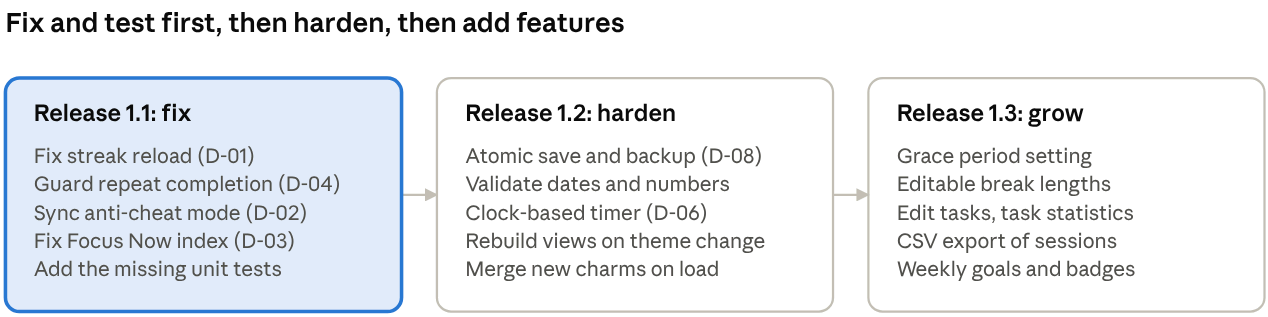
Step	Done when
Run automated tests	All pass
Run the manual cases linked to changed code	All high-priority cases pass
Start with a deleted data file	Seed data created; no console errors
Start with the previous release's data file	Loads without loss
Check the README	Statements match behaviour
Tag the version and archive the folder	Archive created

11.10 Packaging options

For students who do not use Python, the application could be packaged with a freezing tool such as PyInstaller (Windows and Linux) or py2app (macOS) to produce a double-clickable program. The data file path would then need to move from the program folder to the user's application-data folder, because installed programs are often read-only.

12. Future Work, Lessons Learned and Conclusion

12.1 Roadmap



release roadmap · 3 releases

The roadmap puts correctness before features: the first release repairs the defects that undermine the central promises (an honest, working streak; a timer that obeys the chosen mode; a Focus Now button that works) and adds the tests that would have caught them.

12.2 Feature ideas

Idea	Benefit	Effort	Notes
Grace period for short focus loss	Fairer measurement (field already exists)	Small	Ignore losses shorter than grace_period_seconds
Editable break and focus lengths	Flexibility	Small	Two spin boxes in Settings
Edit task	Fix typos and change dates	Small	update_task already exists
Task statistics panel	Motivation	Small	compute_task_statistics already exists
Weekly goals and badges	Longer-term motivation	Medium	New records and reducers
Long-break after four cycles	Authentic Pomodoro technique	Small	Count cycles in the timer

Idea	Benefit	Effort	Notes
Real audio for the radio	Atmosphere	Medium	Standard library only plays system sounds; consider optional extras
Export sessions to CSV	Study reports	Small	Pure function plus a file dialog
Subject editor	Personal courses	Medium	New dialog and reducers
Colour-blind palette and larger text	Accessibility	Medium	New theme and font scale
Break reminders and notifications	Habit support	Medium	Platform-specific code
Sync across devices	Convenience	Large	Needs a server and accounts
Mobile companion	Reach	Large	Different technology

12.3 Architectural evolution

Because the core is independent of the interface, three evolutions are open. The existing Tkinter shell can be kept and improved. A second shell, such as a small web server using the same reducers, could serve phones on the local network. Or the state could move from a JSON file to SQLite if the history grows large. None of these needs changes to the rules.

12.4 Lessons learned

Lesson	Evidence
Separate rules from drawing early	15 percent of the code holds all rules and can be tested in 3 milliseconds
A pure reducer is easy to reason about but still needs tests	The two defects found by running code (D-01 and D-04) were in tested modules
Defensive loading hides data problems	The hasattr filter silently dropped a field

Lesson	Evidence
Duplicate sources of truth drift apart	The two anti-cheat selectors, the threshold sets and the two lists of tasks
Documentation should be checked against the code	Several README claims differ from behaviour
Seed data deserves care	Seed rewards do not match the real formula
Small conveniences carry hidden costs	Using the clock inside reducers makes tests harder

12.5 Conclusion

CozyStudy meets its aim. It is a complete, attractive, dependency-free desktop application that combines a measured focus timer, a reward economy, a task manager, analytics, a calendar and a living room scene, built around an immutable functional core that keeps the rules small, readable and testable. The seven phases described in this report show a coherent path from requirements to a working product.

The review also found a small number of real defects. The most important is that the study streak is silently reset on every reload, so the daily streak, a key motivator, does not accumulate. Others affect the link between tasks and the timer, the anti-cheat mode chosen in Settings, and data safety on save. All are cheap to fix, and the test cases in Chapter 9 show how to guard against them returning. The automated tests pass but cover only about a third of the core's functions, and the interactive layer awaits the designed manual tests.

With the fixes in the first release and the tests in place, CozyStudy would be a robust base for the extensions in the roadmap, and a good worked example of functional design in an everyday desktop program.

Appendix A: Manual Test Execution Record

Use one copy of this sheet per test run. Result is Pass, Fail or Blocked. Enter a defect identifier from Chapter 10 when a case fails.

Field	Entry
Application version or date of files	

Field	Entry
Platform and Python version	
Tester	
Date	
Data file used (fresh seed or supplied)	

Test ID	Result	Defect id	Tester initials	Date
TC-G-01				
TC-G-02				
TC-G-03				
TC-G-04				
TC-G-05				
TC-G-06				
TC-G-07				
TC-G-08				
TC-G-09				
TC-G-10				
TC-G-11				
TC-G-12				
TC-G-13				
TC-G-14				
TC-G-15				
TC-G-16				
TC-G-17				
TC-G-18				
TC-G-19				

Test ID	Result	Defect id	Tester initials	Date
TC-G-20				
TC-G-21				
TC-G-22				
TC-G-23				
TC-G-24				
TC-G-25				
TC-G-26				
TC-G-27				
TC-G-28				
TC-G-29				
TC-G-30				
TC-G-31				
TC-G-32				
TC-G-33				
TC-G-34				
TC-G-35				
TC-G-36				
TC-G-37				
TC-G-38				
TC-G-39				
TC-G-40				
TC-G-41				
TC-G-42				
TC-G-43				
TC-G-44				

Test ID	Result	Defect id	Tester initials	Date
TC-G-45				
TC-G-46				
TC-G-47				
TC-G-48				
TC-G-49				
TC-G-50				
TC-G-51				
TC-G-52				
TC-G-53				
TC-G-54				
TC-G-55				
TC-G-56				
TC-G-57				
TC-G-58				
TC-G-59				
TC-G-60				
TC-G-61				
TC-G-62				
TC-G-63				
TC-G-64				
TC-G-65				
TC-G-66				
TC-G-67				
TC-G-68				
TC-G-69				

Test ID	Result	Defect id	Tester initials	Date
TC-G-70				
TC-G-71				
TC-G-72				
TC-G-73				
TC-G-74				
TC-G-75				
TC-G-76				
TC-G-77				
TC-G-78				

Non-functional record

Test ID	Measured value	Pass or fail	Notes
TC-N-01			
TC-N-02			
TC-N-03			
TC-N-04			
TC-N-05			
TC-N-06			
TC-N-07			
TC-N-08			
TC-N-09			
TC-N-10			
TC-N-11			
TC-N-12			

Appendix B: Requirements Traceability Matrix

Every functional and non-functional requirement is linked to the implementing module (Chapter 4) and to the tests that verify it. "Existing" refers to the six automated tests in test_app.py.

B.1 Functional requirements

Requirement	Implementing module	Verifying tests
FR-01 Default 25-minute focus	timer_view, fp_models	TC-G-09
FR-02 Short and long breaks	timer_view	TC-G-13, TC-G-14
FR-03 MM:SS ring display	fp_utils, timer_view	TC-U-01 to TC-U-04, TC-G-09, TC-G-10
FR-04 Start, pause, resume, reset	timer_view	TC-G-10, TC-G-11, TC-G-12
FR-05 Count focused seconds	timer_view	TC-G-15
FR-06 Count distracted seconds	timer_view	TC-G-15
FR-07 Count window switches	timer_view	TC-G-15, TC-G-17
FR-08 Integrity formula	fp_utils	Existing test 4, TC-U-07, TC-U-08, TC-A-01
FR-09 Strict mode pause	timer_view, main	TC-G-17, TC-G-18, TC-G-24, TC-G-73
FR-10 Alert and chime sounds	timer_view	TC-G-16, TC-G-74
FR-11 Reward scaled by integrity	fp_utils	TC-U-09 to TC-U-18, TC-G-19 to TC-G-21
FR-12 Immutable session record	timer_view, fp_reducers	TC-R-10 to TC-R-12, TC-G-19, TC-G-23

Requirement	Implementing module	Verifying tests
FR-13 Task cycle increment	fp_reducers	TC-R-10, TC-G-22
FR-14 Daily streak	fp_reducers, storage	TC-S-01 to TC-S-06, TC-G-34
FR-15 Room scene	room_canvas	TC-G-27
FR-16 Equipped charms only	room_canvas	TC-G-27, TC-G-29, TC-G-39, TC-G-40
FR-17 Animations	room_canvas	TC-G-32, TC-G-33
FR-18 Cat and lamp clicks	room_canvas	TC-G-28, TC-G-29, TC-G-30
FR-19 Charm listing	wardrobe_dialog	TC-G-35, TC-G-36, TC-G-42
FR-20 Unlock with coins	fp_reducers	TC-R-03 to TC-R-07, TC-G-37, TC-G-38
FR-21 Equip and unequip	fp_reducers	TC-R-08, TC-R-09, TC-G-39, TC-G-40
FR-22 Add task	task_view, fp_reducers	Existing test 3, TC-G-48 to TC-G-50, TC-G-56
FR-23 Task filters	task_view	TC-G-43 to TC-G-47
FR-24 Deadline labels	task_view, fp_models	TC-A-09 to TC-A-11, TC-G-51
FR-25 Focus Now	task_view, main	TC-R-20, TC-G-25
FR-26 Completion bonus	fp_reducers	Existing test 3, TC-R-13 to TC-R-19, TC-G-52, TC-G-53
FR-27 Delete task	fp_reducers, task_view	TC-R-01, TC-R-02, TC-G-54, TC-G-55
FR-28 Analytics	fp_analytics, charts_canvas	Existing test 5, TC-A-01 to TC-A-08, TC-G-57 to TC-G-62
FR-29 Calendar	calendar_view	TC-G-63 to TC-G-70
FR-30 Persistence	storage	Existing test 6, TC-D-01 to TC-D-09, TC-G-01, TC-G-02, TC-G-07
FR-31 Backup and reset	main	TC-G-75 to TC-G-78
FR-32 Themes	theme, main	TC-R-21, TC-G-32, TC-G-71, TC-G-72

Requirement	Implementing module	Verifying tests
FR-33 Keyboard tab shortcuts	main	TC-G-03, TC-G-04

B.2 Non-functional requirements

Requirement	Verifying evidence
NFR-01 Portability	TC-G-08, TC-N-08
NFR-02 No installation step	Inspection: no third-party imports in any module
NFR-03 Maintainability	Inspection; dependency table in Section 5.4
NFR-04 Testability	Existing suite runs in 0.003 seconds without a display
NFR-05 Reliability on bad data	TC-D-02 to TC-D-05, TC-N-06
NFR-06 Data integrity on crash	TC-N-07; not met (D-08)
NFR-07 Usability	TC-G-05, TC-G-06, TC-G-31, TC-G-41, usability plan 9.13
NFR-08 Performance	TC-N-01 to TC-N-05
NFR-09 Privacy	TC-N-12; no network code present
NFR-10 Accessibility	TC-N-11; partly met
NFR-11 Aesthetics	TC-G-32, TC-G-71

Appendix C: Glossary

Term	Meaning
Active focus time	Seconds of a session during which the application window was in the foreground
Anti-cheat	The set of features that measure whether the student stayed in the application during a session

Term	Meaning
after()	Tkinter method that schedules a function to run later on the event loop
Canvas	Tkinter widget for drawing shapes, lines and text
Charm	A decorative item for the room, bought with Star Charms
Combinator	A small function that combines other functions, such as pipe or compose
Controller	The part of the program (main.py) that receives events, calls reducers and refreshes views
Currying	Turning a function of several arguments into a chain of one-argument functions
Dataclass	A Python class that is mostly fields, with an automatically written constructor
Diorama	A small scene; here the illustrated study room
Distracted time	Seconds of a session during which the window was not in the foreground
Event loop	The loop in Tkinter that waits for and dispatches user and timer events
Fold	Combining a sequence of items into one value with an accumulator, also called reduce
Frozen dataclass	A dataclass whose fields cannot be changed after creation
Functional core	The part of the program made of pure functions and immutable data
Focus Integrity	The percentage of session time spent focused in the application
Higher-order function	A function that takes or returns another function
Honest mode	Anti-cheat mode that keeps the timer running when the student leaves and records the time away

Term	Meaning
Immutable	Not changeable after creation
KPI	Key performance indicator; a headline figure such as total focus hours
Pomodoro	A 25-minute focus interval followed by a short break
Pure function	A function whose result depends only on its inputs and which changes nothing outside itself
Reducer	A pure function that takes the current state and an action and returns the next state
Regression test	A test that guards against an old defect coming back
Round trip	Converting data to another form and back and checking nothing is lost
Seed data	Demonstration data created on first run
Serialisation	Converting objects to a storable form such as JSON
Shell (imperative shell)	The part of the program that has side effects: windows, files, sounds
Star Charm	The virtual currency earned by focusing and finishing tasks
State	All of the data that describes the application at one moment
Streak	The number of consecutive days with a completed study session
Strict mode	Anti-cheat mode that pauses the timer when the window loses focus
Tab switch	In this application, one loss of window focus during a running session
Theme token	A named colour in a palette, such as accent or border

Term	Meaning
Toplevel	A Tkinter window separate from the main window, used for dialogs
Tuple	An immutable ordered collection

Appendix D: Function Reference

D.1 fp_utils

Signature	Returns
pipe(value, *functions)	Result of applying the functions left to right
compose(*functions)	A function applying the functions right to left
curry(fn)	A curried version of fn
map_tuple(fn, items)	Tuple of results
filter_tuple(predicate, items)	Tuple of matching items
reduce_iterable(fn, items, initial)	Folded value
find_first(predicate, items, default=None)	First match or default
update_in_tuple(items, predicate, transform)	New tuple with matching items transformed
remove_from_tuple(items, predicate)	New tuple without matching items
clamp(low, high, val)	Value limited to the range
calculate_focus_integrity(active_focus_seconds, total_elapsed_seconds)	Integer 0 to 100
format_seconds_to_min_sec(seconds)	Text MM:SS
calculate_charm_reward(integrity_rate, duration_seconds, is_strict_mode)	Integer, at least 2

D.2 fp_reducers

Signature	Returns
add_task(state, task)	New state
update_task(state, task_id, **updates)	New state
delete_task(state, task_id)	New state
complete_task(state, task_id, today_str=None)	(new state, bonus)
unlock_charm(state, charm_id)	(new state, success)
toggle_equip_charm(state, charm_id)	New state
add_session(state, session)	New state
set_active_task(state, task_id)	New state
update_settings(state, **setting_updates)	New state

D.3 fp_analytics

Signature	Returns
get_today_sessions(sessions, today_str=None)	Tuple of sessions
get_sessions_in_days(sessions, days=7)	Tuple of sessions
compute_aggregate_integrity(sessions)	Dictionary with totals, count, overall_integrity, active_focus_mins, distracted_mins
compute_daily_timeline(sessions, target_date_str=None)	List of dictionaries (id, title, time_label, duration_mins, focus_mins, distracted_mins, tab_switches, integrity, color, status_text)
compute_7_day_trends(sessions)	List of seven dictionaries (date_str, day_label, focus_mins, distracted_mins, tab_switches, integrity, session_count)

Signature	Returns
compute_task_statistics(tasks)	Dictionary (total, completed, overdue, due_today, high_priority, completion_rate)

D.4 storage

Signature	Returns
get_default_subjects()	Tuple of five Subject records
get_default_charms()	Tuple of twelve Charm records
get_default_tasks()	Tuple of five Task records
get_default_sessions()	Tuple of six FocusSession records
get_default_state()	Seed AppState
state_to_dict(state)	JSON-compatible dictionary
dict_to_state(data)	AppState, or defaults on error
load_state()	AppState from file or seed
save_state(state)	True or False

D.5 Models with behaviour

Signature	Returns
Task.is_overdue(reference_date_str=None)	True if not completed and due before the date
Task.is_due_today(reference_date_str=None)	True if not completed and due on the date

Appendix E: File Inventory

File	Type	Purpose
main.py	Source	Application entry point and controller
fp_models.py	Source	Immutable data models
fp_utils.py	Source	Functional combinators and formulas
fp_reducers.py	Source	Pure state transitions
fp_analytics.py	Source	Analytics pipelines
storage.py	Source	Persistence and seed data
theme.py	Source	Colour palettes and fonts
room_canvas.py	Source	Room diorama
timer_view.py	Source	Focus timer and anti-cheat
charts_canvas.py	Source	Analytics charts
task_view.py	Source	Task manager
calendar_view.py	Source	Calendar
wardrobe_dialog.py	Source	Charm shop
test_app.py	Test	Automated tests
run.sh	Script	Launcher
README.md	Document	User-facing description
cozystudy_data.json	Data	Saved state of one user

Appendix F: Worked Examples

F.1 Reward calculation

A student completes a 25-minute honest-mode session. The window was away for 180 seconds.

Quantity	Calculation	Value
Active focus seconds	1500 minus 180	1320
Integrity	round(1320 / 1500 x 100) = round(88.0)	88
Base reward	1500 // 100	15
Multiplier	Integrity 80 to 89, honest	1.0
Reward	max(2, round(15 x 1.0))	15
Same session in strict mode	multiplier 1.2, round(18.0)	18

F.2 Task completion bonus

Task	Due	Priority	Calculation	Bonus
Problem set	Tomorrow	High	20 + 15 + 10	45
Essay draft	Next week	Medium	20 + 15	35
Reading	Yesterday	Low	20	20
Lab report	Yesterday	High	20 + 10	30

F.3 Streak sequences

Day	Studied	Intended streak	Notes
Monday	Yes	1	First day
Tuesday	Yes	2	Yesterday matched
Wednesday	Yes	3	
Thursday	No	3	No change until next session
Friday	Yes	1	Gap of one day resets; best remains 3

With defect D-01 present, restarting the program between any two of these days freezes the streak at its stored value.

F.4 Donut geometry for the seed data

Focus is 141.7 minutes and off-tab is 13.3 minutes, a total of 155. The green arc covers 141.7 / 155 x 360, about 329.1 degrees, and the red remainder about 30.9 degrees. The centre text shows 91 percent because the integrity is computed from seconds (8,500 of 9,300).

F.5 Timer ring geometry

The ring has a radius of 112 pixels (0.40 of the 280-pixel canvas). After 10 minutes of a 25-minute session the elapsed fraction is 600 / 1500, or 0.4, so the arc extent is minus 144 degrees, drawn clockwise from twelve o'clock. At 25:00 the arc is absent; at 00:00 it is a full circle.

F.6 Calendar layout for October 2026

October 1, 2026 is a Thursday, so the Python month matrix has five weeks: the first week begins with three blank cells, then 1 to 4; the following weeks run 5 to 11, 12 to 18 and 19 to 25; the last week is 26 to 31 followed by one blank cell. Today, Friday 16 October, sits in the third row.

F.7 Deadline labels for the supplied data file on 16 October 2026

Task	Due date	Status	Label shown
Linear Algebra Problem Set 4	2026-09-14	todo	Overdue (2026-09-14)
Literature Comparative Synthesis Essay	2026-09-17	in progress	Overdue (2026-09-17)
Design Pastel Palette and Room Moodboard	2026-09-16	todo	Overdue (2026-09-16)
Review Cellular Respiration Diagram and Enzymes	2026-09-19	completed	Completed

This is the situation of a student who has not opened the program for a month, and illustrates why the on-time bonus rule matters: completing any of these three now would earn only the base bonus (20, or 30 for the high-priority task).

End of report.